COMPLETE SYSTEM ARCHITECTURE & SOURCE CODE

Speech Emotion Recognition (SER)

End-to-End Machine Learning Audio Processing Pipeline, REST API Server, & Modern Responsive Web Application

PRIMARY ML MODELS
MLP Neural Net, SVM, Random Forest, GBDT

ACOUSTIC FEATURE VECTOR
182 Dimensions (MFCC, Chroma, Mel, Tonnetz)

AUDIO SAMPLE RATE / FORMAT
22,050 Hz Mono / WAV (RAVDESS Corpus)

TARGET EMOTION CLASSES
8 Emotions (Happy, Sad, Angry, Fear, etc.)

BACKEND & API FRAMEWORK
FastAPI / Uvicorn (Asynchronous REST)

FRONTEND & AUDIO ENGINE
Web Audio API / HTML5 Canvas / Modern CSS

TOTAL CODEBASE SIZE
4,835 Lines of Code across 17 Files

VERIFICATION & TESTS
Pytest Automated Pipeline Suite

Project Repository: speech-emotion-detection
License / Usage: Academic, Research & Commercial Ready

Generated Documentation • Section-by-Section Code Reference

Table of Contents & Architecture Index

1. Project Overview & System Architecture

High-level design, audio signal processing flow, ML models, and tech stack

2. Machine Learning & Signal Processing Pipeline

Audio ingestion, preprocessing, acoustic feature extraction, and model training (src/)

`src/config.py` — config.py	90 LOC
`src/dataset.py` — dataset.py	212 LOC
`src/preprocessing.py` — preprocessing.py	333 LOC
`src/feature_extraction.py` — feature_extraction.py	245 LOC
`src/train.py` — train.py	656 LOC
`src/analyze_dataset.py` — analyze_dataset.py	289 LOC
`src/download_dataset.py` — download_dataset.py	211 LOC

3. Backend API Server

FastAPI REST API, asynchronous audio inference, CORS, and health endpoints (backend/)

`backend/main.py` — main.py	448 LOC

4. Frontend Web Application

Modern responsive dashboard, Web Audio API recording, real-time waveform visualizer (frontend/)

`frontend/index.html` — index.html	297 LOC
`frontend/style.css` — style.css	573 LOC
`frontend/app.js` — app.js	526 LOC

5. Testing & Verification Suite

Automated unit and integration test coverage for signal processing and ML pipeline (tests/)

`tests/test_pipeline.py` — test_pipeline.py	544 LOC

6. Model Configuration, Dependencies & Launch Scripts

Serialized model metadata, feature dimension configurations, dependencies, and runner scripts

`models/model_metadata.json` — model_metadata.json	24 LOC
`models/features_config.json` — features_config.json	10 LOC
`requirements.txt` — requirements.txt	40 LOC
`start.py` — start.py	193 LOC
`run.bat` — run.bat	144 LOC

Section 1: Project Overview & System Architecture

Comprehensive architectural breakdown of the Speech Emotion Recognition (SER) system

1.1 System Architecture & Audio Pipeline Workflow

The Speech Emotion Recognition system processes human vocal audio signals and classifies the underlying affective emotion state into 8 distinct emotional categories: **Neutral, Calm, Happy, Sad, Angry, Fearful, Disgust, and Surprised**. The end-to-end architecture is partitioned into three decoupled tiers: the Machine Learning pipeline, the FastAPI asynchronous backend, and the responsive HTML5/JS Web Audio client.

1. **Audio Capture & Upload Tier (Frontend Web Audio)**
   Microphone input recorded directly via browser Web Audio API & MediaRecorder at 16-bit PCM, or pre-recorded WAV/MP3 drag-and-drop file upload. Real-time visual feedback rendered on HTML5 canvas via frequency/time-domain analyzer node.

2. **Signal Preprocessing & Normalization Tier (Librosa / NumPy)**
   Audio decoded and resampled to 22,050 Hz mono. Silent segments trimmed using top-db energy thresholding. Signals padded or truncated to a fixed 3.0 second duration (66,150 samples). Signal amplitudes normalized to unit peak and RMS energy.

3. **Acoustic Feature Extraction Tier (182-Dimensional Vector)**
   Extracts time-frequency representations: 40 MFCCs (mean & std), 12 Chroma STFT pitch classes (mean & std), 40 Mel Spectrogram filterbanks (mean), 7 Spectral Contrast octaves (mean & std), 6 Tonnetz tonal centroid features (mean & std), Zero-Crossing Rate (mean & std), and Root-Mean-Square energy (mean & std).

4. **Machine Learning Inference Tier (Scikit-Learn)**
   Feature vectors normalized using fitted StandardScaler. Fed into trained Multi-Layer Perceptron (MLP) Classifier (or SVM/Random Forest). Generates calibrated probability distributions across all 8 emotion classes.

5. **REST API & Interactive Visualization Tier**
   FastAPI asynchronous worker serializes predictions to JSON response. Client dynamically renders primary emotion card, sentiment emoji, confidence percentage progress bar, and comprehensive probability breakdown bars.

1.2 Acoustic Feature Breakdown (182 Features)

Feature Representation	Dimensions	Statistical Aggregation	Physical / Perceptual Significance
MFCCs (Mel-Frequency Cepstral Coefficients)	80 (40 mean + 40 std)	Mean & Standard Deviation	Vocal tract timbre, formant frequencies, and speech phonetic structure.
Chroma STFT	24 (12 mean + 12 std)	Mean & Standard Deviation	Pitch class energy distribution across 12 chromatic musical semitones.
Mel Spectrogram Filterbanks	40 (40 mean)	Mean over frames	Perceptually scaled auditory frequency energy distribution.
Spectral Contrast	14 (7 mean + 7 std)	Mean & Standard Deviation	Relative difference between harmonic peaks and spectral valleys per octave.
Tonnetz (Tonal Centroid Features)	12 (6 mean + 6 std)	Mean & Standard Deviation	Harmonic pitch relations projected onto a 6D tonal space (fifths, thirds).
Zero Crossing Rate (ZCR)	2 (1 mean + 1 std)	Mean & Standard Deviation	Rate of sign changes in audio signal; measures unvoiced speech & harshness.
Root Mean Square (RMS) Energy	2 (1 mean + 1 std)	Mean & Standard Deviation	Temporal loudness and vocal energy envelope.
Total Combined Feature Vector	**182 Features**	StandardScaler Normalization	Unified representation capturing spectral, timbral, harmonic, and rhythmic cues.

2. Machine Learning & Signal Processing Pipeline

Audio ingestion, preprocessing, acoustic feature extraction, and model training (src/)

PY **src/config.py** 90 Lines • 4,068 bytes

Description & Role: Central configuration for audio sample rates (22,050 Hz), duration (3.0s), acoustic feature extraction parameters (MFCCs, Chroma, Mel Spectrogram, Spectral Contrast, Tonnetz, ZCR, RMS), model hyperparameters, file paths, and emotion label dictionaries.

```python
"""
config.py - Central configuration for the Speech Emotion Detection project.
All paths, hyperparameters, and constants are defined here so nothing is hard-coded
elsewhere in the codebase.
"""

import os
from pathlib import Path

# ── Project root ──────────────────────────────────────────────────────────────
PROJECT_ROOT = Path(__file__).resolve().parent.parent  # speech-emotion-detection/

# ── Data paths ────────────────────────────────────────────────────────────────
DATA_DIR        = PROJECT_ROOT / "data"
RAW_DIR         = DATA_DIR / "raw" / "RAVDESS"
PROCESSED_DIR   = DATA_DIR / "processed"
FEATURES_DIR    = DATA_DIR / "features"

# ── Model paths ───────────────────────────────────────────────────────────────
MODELS_DIR      = PROJECT_ROOT / "models"
MODEL_PATH      = MODELS_DIR / "speech_emotion_model.pkl"
SCALER_PATH     = MODELS_DIR / "scaler.pkl"
LABEL_ENCODER_PATH = MODELS_DIR / "label_encoder.pkl"
METADATA_PATH   = MODELS_DIR / "model_metadata.json"
FEATURES_CONFIG_PATH = MODELS_DIR / "features_config.json"

# ── Results paths ─────────────────────────────────────────────────────────────
RESULTS_DIR     = PROJECT_ROOT / "results"
CONFUSION_MATRIX_PATH   = RESULTS_DIR / "confusion_matrix.png"
CLASSIFICATION_REPORT_PATH = RESULTS_DIR / "classification_report.txt"
METRICS_JSON_PATH       = RESULTS_DIR / "metrics.json"
MODEL_COMPARISON_PATH   = RESULTS_DIR / "model_comparison.csv"

# ── Audio preprocessing settings ──────────────────────────────────────────────
SAMPLE_RATE     = 22050          # Target sample rate (Hz) — librosa default
AUDIO_DURATION  = 3.0            # Fixed duration (seconds) for feature extraction
                                 # RAVDESS clips are ~3-4s; we pad/trim to 3s
N_MFCC          = 40             # Number of MFCC coefficients
N_MELS          = 128            # Mel filter banks
HOP_LENGTH      = 512            # Hop length for STFT
N_FFT           = 2048           # FFT window size
TOP_DB          = 20             # Threshold for silence trimming (dB below peak)

# ── Feature extraction settings ───────────────────────────────────────────────
FEATURE_CONFIG = {
    "mfcc":             True,    # 40 MFCCs (mean + std = 80 values)
    "mfcc_delta":       True,    # Delta MFCCs (80 values)
    "mel_spectrogram":  False,   # Skipped: high-dim; MFCCs already capture it
    "zero_crossing_rate": True,  # ZCR mean + std (2 values)
    "rms_energy":       True,    # RMS mean + std (2 values)
    "spectral_centroid": True,   # Spectral centroid mean + std (2 values)
    "spectral_rolloff":  True,   # Spectral rolloff mean + std (2 values)
    "chroma":           True,    # Chroma features mean + std (24 values)
}
# Total feature vector size: 80 + 80 + 2 + 2 + 2 + 2 + 24 = 192 features

# ── Train/Val/Test split ──────────────────────────────────────────────────────
TEST_SIZE       = 0.15   # 15% for final testing
VAL_SIZE        = 0.15   # 15% for validation (from training set)
RANDOM_SEED     = 42     # Fixed seed for reproducibility

# ── RAVDESS emotion mapping ───────────────────────────────────────────────────
# RAVDESS file naming convention: 3rd segment is emotion code
# 01=neutral, 02=calm, 03=happy, 04=sad, 05=angry, 06=fearful, 07=disgust, 08=surprised
RAVDESS_EMOTION_MAP = {
    "01": "Neutral",
    "02": "Calm",
    "03": "Happy",
    "04": "Sad",
    "05": "Angry",
    "06": "Fearful",
    "07": "Disgust",
    "08": "Surprised",
}

# All 8 supported emotion classes (in consistent order)
EMOTION_CLASSES = [
    "Neutral", "Calm", "Happy", "Sad",
```

```
    "Angry", "Fearful", "Disgust", "Surprised"
]

# ── Backend settings ──────────────────────────────────────────────
BACKEND_HOST    = "127.0.0.1"
BACKEND_PORT    = 8000
MIN_AUDIO_DURATION = 0.5   # Minimum acceptable recording duration (seconds)
MAX_AUDIO_SIZE_MB  = 10    # Maximum upload file size (MB)

# ── Ensure output directories exist ──────────────────────────────
for _dir in [PROCESSED_DIR, FEATURES_DIR, MODELS_DIR, RESULTS_DIR]:
    _dir.mkdir(parents=True, exist_ok=True)
```

PY **src/dataset.py** 212 Lines • 6,972 bytes

Description & Role: Dataset parser and loader for RAVDESS audio dataset. Implements filename convention parsing (modality, vocal channel, emotion index, intensity, statement, repetition, actor ID, gender), dataset verification, and stratified train/val/test split generation.

```python
"""
dataset.py - Dataset loading and analysis for RAVDESS Speech Emotion Dataset.

RAVDESS Dataset:
- Full name: Ryerson Audio-Visual Database of Emotional Speech and Song
- URL: https://zenodo.org/record/1188976
- License: Creative Commons Attribution (CC BY 4.0)
- Speakers: 24 professional actors (12 male, 12 female)
- Emotions: 8 (Neutral, Calm, Happy, Sad, Angry, Fearful, Disgust, Surprised)
- Files: 1440 audio files (speech only, two intensity levels)

File naming convention:
  Modality-VocalChannel-Emotion-Intensity-Statement-Repetition-Actor.wav
  Example: 03-01-05-01-01-01-01.wav
  Position 3 (0-indexed 2) = Emotion code (01-08)
  Position 7 = Actor ID (01-24)

Dataset split strategy:
- We use SPEAKER-INDEPENDENT splitting.
- Speakers are randomly divided into train/val/test groups.
- This prevents data leakage: the model never sees a training speaker
  in the test set, making evaluation more realistic.
- Split: 70% train / 15% val / 15% test (approximately)
- Train speakers: 17, Val speakers: 4, Test speakers: 3 (out of 24)
"""

import logging
import json
import numpy as np
import pandas as pd
from pathlib import Path
from typing import Dict, List, Tuple, Optional
import sys

sys.path.insert(0, str(Path(__file__).resolve().parent))
from config import (
    RAW_DIR, RAVDESS_EMOTION_MAP, EMOTION_CLASSES, RANDOM_SEED,
    TEST_SIZE, VAL_SIZE
)

logger = logging.getLogger(__name__)


def parse_ravdess_filename(filepath: Path) -> Optional[Dict]:
    """
    Parse a RAVDESS filename and return metadata.

    RAVDESS naming: Modality-VocalChannel-Emotion-Intensity-Statement-Repetition-Actor.wav
    Segments are separated by '-'.

    Returns dict with: emotion_code, emotion_label, actor_id, intensity, filepath
    Returns None if filename doesn't match expected RAVDESS pattern.
    """
    try:
        parts = filepath.stem.split('-')
        if len(parts) != 7:
            return None

        modality     = parts[0]   # 01=audio-only, 02=audio-video
        vocal_channel = parts[1]  # 01=speech, 02=song
        emotion_code = parts[2]   # 01-08
        intensity    = parts[3]   # 01=normal, 02=strong
        statement    = parts[4]   # 01="Kids are talking...", 02="Dogs are sitting..."
        repetition   = parts[5]   # 01 or 02
        actor_id     = int(parts[6])  # 01-24

        # Only use speech modality (not song)
        if vocal_channel != "01":
            return None

        emotion_label = RAVDESS_EMOTION_MAP.get(emotion_code)
        if emotion_label is None:
            logger.warning(f"Unknown emotion code {emotion_code} in {filepath.name}")
            return None

        return {
            "filepath":     str(filepath),
            "emotion_code": emotion_code,
```

```
            "emotion":      emotion_label,
            "actor_id":     actor_id,
            "intensity":    intensity,
            "statement":    statement,
            "repetition":   repetition,
            "modality":     modality,
        }
    except Exception as e:
        logger.debug(f"Could not parse {filepath.name}: {e}")
        return None


def load_dataset_info(data_dir: Path = RAW_DIR) -> pd.DataFrame:
    """
    Scan the RAVDESS directory and build a DataFrame with file metadata.

    RAVDESS structure (after extraction):
      data/raw/RAVDESS/
        Actor_01/
          03-01-01-01-01-01-01.wav
          03-01-02-01-01-01-01.wav
          ...
        Actor_02/
          ...

    Returns:
        DataFrame with columns: filepath, emotion, actor_id, intensity, etc.
        Returns empty DataFrame if dataset not found.
    """
    audio_files = list(data_dir.rglob("*.wav"))

    if not audio_files:
        logger.error(
            f"No .wav files found in {data_dir}.\n"
            f"Please download RAVDESS from: https://zenodo.org/record/1188976\n"
            f"and extract to: {data_dir}"
        )
        return pd.DataFrame()

    records = []
    skipped = 0
    for fpath in audio_files:
        info = parse_ravdess_filename(fpath)
        if info is not None:
            records.append(info)
        else:
            skipped += 1

    if skipped > 0:
        logger.info(f"Skipped {skipped} files (songs or unrecognized format).")

    df = pd.DataFrame(records)
    logger.info(f"Dataset loaded: {len(df)} audio files across {df['actor_id'].nunique()} speakers.")
    return df


def get_speaker_split(
    df: pd.DataFrame,
    test_size: float = TEST_SIZE,
    val_size: float = VAL_SIZE,
    random_seed: int = RANDOM_SEED,
) -> Tuple[List[int], List[int], List[int]]:
    """
    Split speakers into train/val/test groups (speaker-independent split).

    Why speaker-independent?
    - If speaker 5's files appear in both train and test, the model
      learns that speaker's voice pattern, not the emotion.
    - Speaker-independent split forces generalization across voices.

    Returns:
        (train_actor_ids, val_actor_ids, test_actor_ids)
    """
    rng = np.random.default_rng(random_seed)
    actor_ids = sorted(df['actor_id'].unique())
    n_actors = len(actor_ids)

    shuffled = list(rng.permutation(actor_ids))

    n_test = max(1, round(n_actors * test_size))
    n_val  = max(1, round(n_actors * val_size))
    n_train = n_actors - n_test - n_val

    train_actors = shuffled[:n_train]
    val_actors   = shuffled[n_train:n_train + n_val]
    test_actors  = shuffled[n_train + n_val:]
```

```
    logger.info(
        f"Speaker split -> Train: {sorted(train_actors)} ({len(train_actors)} speakers), "
        f"Val: {sorted(val_actors)} ({len(val_actors)} speakers), "
        f"Test: {sorted(test_actors)} ({len(test_actors)} speakers)"
    )
    return train_actors, val_actors, test_actors


def split_dataset(
    df: pd.DataFrame,
    random_seed: int = RANDOM_SEED,
) -> Tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """
    Split the dataset into train/val/test DataFrames using speaker-independent split.

    Returns:
        (train_df, val_df, test_df)
    """
    train_actors, val_actors, test_actors = get_speaker_split(df, random_seed=random_seed)

    train_df = df[df['actor_id'].isin(train_actors)].reset_index(drop=True)
    val_df   = df[df['actor_id'].isin(val_actors)].reset_index(drop=True)
    test_df  = df[df['actor_id'].isin(test_actors)].reset_index(drop=True)

    logger.info(
        f"Split sizes -> Train: {len(train_df)}, Val: {len(val_df)}, Test: {len(test_df)}"
    )
    return train_df, val_df, test_df


def verify_dataset(data_dir: Path = RAW_DIR) -> bool:
    """
    Check whether RAVDESS dataset exists and has the expected structure.
    Returns True if dataset is ready, False otherwise.
    """
    if not data_dir.exists():
        return False

    wav_files = list(data_dir.rglob("*.wav"))
    if len(wav_files) < 100:
        logger.warning(
            f"Found only {len(wav_files)} .wav files in {data_dir}. "
            f"RAVDESS should have ~1440 speech files."
        )
        return False

    return True
```

PY **src/preprocessing.py** 333 Lines • 11,015 bytes

Description & Role: Audio signal preprocessing and data augmentation module. Implements non-silent audio trimming (librosa.effects.trim), duration normalization (padding/truncating), DC-offset removal, RMS amplitude normalization, and data augmentation (white noise injection, pitch shifting, time stretching, random time shifting).

```python
"""
preprocessing.py - Audio preprocessing pipeline for Speech Emotion Detection.

This EXACT same pipeline is used for:
  1. Training audio files
  2. Test audio files
  3. Live microphone recordings from the backend

Any change here affects ALL three — keeping training and inference consistent.
"""

import logging
import numpy as np
import soundfile as sf
import librosa
from pathlib import Path
from typing import Optional, Tuple

# Import config from same src/ package
import sys
sys.path.insert(0, str(Path(__file__).resolve().parent))
from config import SAMPLE_RATE, AUDIO_DURATION, TOP_DB

logger = logging.getLogger(__name__)


def transcode_to_wav(input_path: Path, output_wav_path: Path) -> bool:
    """Convert any audio file format to 16-bit PCM WAV using imageio_ffmpeg."""
    try:
        import subprocess
        import imageio_ffmpeg
        ffmpeg_exe = imageio_ffmpeg.get_ffmpeg_exe()
        cmd = [
            ffmpeg_exe, "-y",
            "-i", str(input_path),
            "-ac", "1",
            "-ar", "22050",
            "-sample_fmt", "s16",
            str(output_wav_path)
        ]
        result = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE, timeout=15)
        return result.returncode == 0 and output_wav_path.exists()
    except Exception as e:
        logger.error(f"Transcoding failed for {input_path}: {e}")
        return False


def load_audio(
    audio_path: str,
    target_sr: int = SAMPLE_RATE,
) -> Tuple[Optional[np.ndarray], int]:
    """
    Load an audio file and return (waveform, sample_rate).

    Steps:
    1. Read the file with librosa.
    2. If direct read fails (e.g. webm/opus on Windows), transcode to wav via ffmpeg.
    3. Force mono and resample to target_sr.

    Returns (None, 0) if loading fails.
    """
    audio_path = Path(audio_path)
    try:
        import soundfile as sf
        import scipy.signal as sps
        y, file_sr = sf.read(str(audio_path), dtype='float32')
        if y.ndim > 1:
            y = np.mean(y, axis=1)
        if file_sr != target_sr:
            y = sps.resample_poly(y, target_sr, file_sr).astype(np.float32)
        return y, target_sr
    except Exception:
        pass

    try:
        y, sr = librosa.load(str(audio_path), sr=target_sr, mono=True)
```

```
            return y, sr
        except Exception as e:
            logger.info(f"Direct load failed for {audio_path.name} ({e}); attempting ffmpeg transcoding...")
            import tempfile
            with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as tmp_wav:
                tmp_wav_path = Path(tmp_wav.name)
            try:
                if transcode_to_wav(audio_path, tmp_wav_path):
                    y, sr = librosa.load(str(tmp_wav_path), sr=target_sr, mono=True)
                    return y, sr
                else:
                    logger.error(f"Failed to load audio {audio_path}: {e}")
                    return None, 0
            except Exception as e2:
                logger.error(f"Transcoded load failed for {audio_path}: {e2}")
                return None, 0
            finally:
                if tmp_wav_path.exists():
                    try:
                        tmp_wav_path.unlink()
                    except Exception:
                        pass


def validate_audio(y: np.ndarray, sr: int, min_duration: float = 0.3) -> Tuple[bool, str]:
    """
    Validate a loaded audio array.

    Checks:
    - Array is not None or empty
    - Duration is above minimum threshold
    - Audio is not pure silence

    Returns (is_valid, reason_if_invalid)

    Note: The RMS silence threshold is intentionally lenient (1e-4) because
    browser-recorded WebM/Opus audio often decodes at low amplitude.
    """
    if y is None or len(y) == 0:
        return False, "Audio array is empty or None."

    duration = len(y) / sr
    if duration < min_duration:
        return False, f"Audio too short: {duration:.2f}s (minimum {min_duration}s)."

    # Check if audio is pure silence (all zeros or near-zero)
    rms = np.sqrt(np.mean(y ** 2))
    logger.debug(f"Audio RMS level: {rms:.6f}, duration: {duration:.2f}s")
    if rms < 1e-4:
        return False, f"Audio appears to be silent (RMS={rms:.2e}, too low)."

    return True, "OK"


def remove_silence(y: np.ndarray, sr: int, top_db: int = TOP_DB) -> np.ndarray:
    """
    Fast vectorized silence trimming using energy thresholding with safety margins.
    """
    MIN_KEPT_DURATION = 0.5  # seconds
    if len(y) <= int(sr * MIN_KEPT_DURATION):
        return y

    try:
        max_amp = np.max(np.abs(y))
        thresh = max(max_amp * 0.05, 0.005)
        non_silent = np.where(np.abs(y) > thresh)[0]
        if len(non_silent) > sr * MIN_KEPT_DURATION:
            start = max(0, non_silent[0] - int(sr * 0.05))
            end = min(len(y), non_silent[-1] + int(sr * 0.05))
            return y[start:end]
    except Exception:
        pass
    return y


def normalize_audio(y: np.ndarray) -> np.ndarray:
    """
    Peak normalize audio to the range [-1.0, 1.0].

    This makes feature extraction independent of recording volume,
    which is important because different microphones/recordings have
    different loudness levels.
    """
    max_val = np.max(np.abs(y))
    if max_val > 0:
        return y / max_val
```

```python
    return y


def pad_or_trim(y: np.ndarray, sr: int, duration: float = AUDIO_DURATION) -> np.ndarray:
    """
    Ensure all audio clips are exactly `duration` seconds long.

    - If shorter: zero-pad at the end.
    - If longer: trim to the first `duration` seconds.

    Why fixed length? Machine learning models need consistent input size.
    Feature extraction (MFCC) can work with variable length, but using
    fixed length ensures consistent feature dimensions across all samples.
    """
    target_length = int(sr * duration)
    current_length = len(y)

    if current_length < target_length:
        # Pad with zeros at the end
        padding = target_length - current_length
        y = np.pad(y, (0, padding), mode='constant', constant_values=0)
    elif current_length > target_length:
        # Trim to target length
        y = y[:target_length]

    return y


def preprocess_audio(
    audio_path: str,
    target_sr: int = SAMPLE_RATE,
    duration: float = AUDIO_DURATION,
    apply_silence_removal: bool = True,
    apply_normalization: bool = True,
) -> Tuple[Optional[np.ndarray], int]:
    """
    MAIN PREPROCESSING FUNCTION — used for training, testing, and live audio.

    Pipeline:
    1. Load audio (mono, resampled to target_sr)
    2. Validate audio
    3. Remove silence (optional)
    4. Normalize amplitude (optional)
    5. Pad or trim to fixed duration

    Args:
        audio_path: Path to audio file (WAV, FLAC, OGG, etc.)
        target_sr: Target sample rate in Hz
        duration: Fixed duration in seconds
        apply_silence_removal: Whether to trim silence
        apply_normalization: Whether to peak-normalize

    Returns:
        (processed_waveform, sample_rate) or (None, 0) on failure
    """
    # Step 1: Load
    y, sr = load_audio(audio_path, target_sr)
    if y is None:
        return None, 0

    # Step 2: Validate
    is_valid, reason = validate_audio(y, sr)
    if not is_valid:
        logger.warning(f"Audio validation failed for {audio_path}: {reason}")
        return None, 0

    # Step 3: Remove silence
    if apply_silence_removal:
        y = remove_silence(y, sr)

    # Step 4: Normalize
    if apply_normalization:
        y = normalize_audio(y)

    # Step 5: Pad or trim to fixed length
    y = pad_or_trim(y, sr, duration)

    return y, sr


def preprocess_audio_bytes(
    audio_bytes: bytes,
    original_format: str = "wav",
    target_sr: int = SAMPLE_RATE,
    duration: float = AUDIO_DURATION,
) -> Tuple[Optional[np.ndarray], int]:
```

```
    """
    Preprocess audio from raw bytes (used in the backend for uploaded files).

    The browser may send WebM/Opus audio. This function:
    1. Writes bytes to a temporary file with the correct extension.
    2. Attempts to load with librosa directly (works for WAV/FLAC/OGG).
    3. If that fails (common for WebM/Opus on Windows), transcodes to WAV via
       imageio_ffmpeg and retries.
    4. Applies the standard preprocessing pipeline.

    Args:
        audio_bytes: Raw audio bytes from HTTP upload
        original_format: Format hint (e.g., 'webm', 'wav', 'ogg')
        target_sr: Target sample rate
        duration: Fixed duration

    Returns:
        (processed_waveform, sample_rate) or (None, 0) on failure
    """
    import tempfile
    import os

    if not audio_bytes:
        logger.error("preprocess_audio_bytes: received empty bytes.")
        return None, 0

    suffix = f".{original_format}" if not original_format.startswith('.') else original_format

    # Write the raw bytes to a temp file
    tmp_input_path = None
    tmp_wav_path = None
    try:
        with tempfile.NamedTemporaryFile(suffix=suffix, delete=False) as tmp:
            tmp.write(audio_bytes)
            tmp_input_path = tmp.name

        logger.debug(
            f"preprocess_audio_bytes: wrote {len(audio_bytes)} bytes to {tmp_input_path} "
            f"(format={original_format})"
        )

        # --- Attempt 1: direct preprocessing (works for WAV, FLAC, OGG) ---
        result = preprocess_audio(tmp_input_path, target_sr, duration)
        if result[0] is not None:
            logger.debug("preprocess_audio_bytes: direct load succeeded.")
            return result

        # --- Attempt 2: ffmpeg transcode to WAV, then preprocess ---
        # This is the common path for browser WebM/Opus on Windows where
        # librosa/soundfile can't decode WebM without system ffmpeg.
        logger.info(
            f"Direct load failed for {suffix} audio; "
            "attempting ffmpeg transcode to WAV..."
        )
        with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as tmp_wav:
            tmp_wav_path = tmp_wav.name

        if transcode_to_wav(Path(tmp_input_path), Path(tmp_wav_path)):
            result = preprocess_audio(tmp_wav_path, target_sr, duration)
            if result[0] is not None:
                logger.info("preprocess_audio_bytes: ffmpeg transcode + load succeeded.")
                return result
            else:
                logger.error(
                    "preprocess_audio_bytes: ffmpeg transcode succeeded but "
                    "preprocessing still failed (audio may be silent or too short)."
                )
        else:
            logger.error(
                "preprocess_audio_bytes: ffmpeg transcode failed. "
                "Ensure imageio_ffmpeg is installed: pip install imageio-ffmpeg"
            )

        return None, 0

    except Exception as e:
        logger.error(f"preprocess_audio_bytes: unexpected error: {e}", exc_info=True)
        return None, 0
    finally:
        for path in [tmp_input_path, tmp_wav_path]:
            if path:
                try:
                    os.unlink(path)
                except Exception:
                    pass
```

PY **src/feature_extraction.py** 245 Lines • 9,727 bytes

Description & Role: Comprehensive acoustic feature extraction engine generating 182-dimensional feature vectors per audio sample. Extracts MFCCs (40 mean + 40 std), Chroma STFT (12 mean + 12 std), Mel Spectrogram (40 mean), Spectral Contrast (7 mean + 7 std), Tonnetz (6 mean + 6 std), Zero Crossing Rate (1 mean + 1 std), and RMS Energy (1 mean + 1 std) along with caching mechanisms.

```
"""
feature_extraction.py - ENHANCED Feature Extraction for Speech Emotion Detection.

Features extracted (~560 total):
──────────────────────────────────────────────────────────────────
Feature                  | Dim  | Why it helps
──────────────────────────────────────────────────────────────────
MFCC (40 coeff)          | 80   | Vocal tract shape — primary SER feature
MFCC Delta               | 80   | Rate of change — speech dynamics
MFCC Delta-Delta         | 80   | Acceleration — captures emotional transitions
Zero Crossing Rate       | 2    | Voiced vs unvoiced segments
RMS Energy               | 2    | Loudness — angry vs calm/sad
Spectral Centroid        | 2    | Brightness — different per emotion
Spectral Rolloff         | 2    | Tone characteristic
Spectral Bandwidth       | 2    | Spread of spectrum
Spectral Contrast        | 14   | Peak-valley per sub-band — very discriminative
Chroma STFT              | 24   | Pitch class energy — tonal content
Tonnetz                  | 12   | Tonal centroid — harmony relationships
Fundamental Frequency F0 | 4    | Pitch mean/std/min/max — #1 emotion correlate
Mel Spectrogram Stats    | 256  | 128 mel bands × (mean+std) — rich texture
──────────────────────────────────────────────────────────────────
Total feature vector: ~560 values (vs 192 before — 3x richer)
"""

import logging
import numpy as np
import librosa
from pathlib import Path
from typing import Optional
import sys

sys.path.insert(0, str(Path(__file__).resolve().parent))
from config import (
    SAMPLE_RATE, N_MFCC, N_MELS, HOP_LENGTH, N_FFT, FEATURE_CONFIG
)
from preprocessing import preprocess_audio

logger = logging.getLogger(__name__)


def extract_features_from_waveform(
    y: np.ndarray,
    sr: int = SAMPLE_RATE,
) -> Optional[np.ndarray]:
    """
    Extract a rich fixed-size feature vector from a preprocessed audio waveform.

    Both training and live inference call this — identical features ensure
    training/inference consistency.

    Args:
        y:  Audio waveform (numpy array, mono, fixed length)
        sr: Sample rate in Hz

    Returns:
        1D numpy array of features (~560 values), or None on failure.
    """
    try:
        features = []

        # ── 1. MFCC mean + std (80 values) ──────────────────────────
        mfcc = librosa.feature.mfcc(
            y=y, sr=sr, n_mfcc=N_MFCC, n_fft=N_FFT, hop_length=HOP_LENGTH
        )
        features.extend(np.mean(mfcc, axis=1))
        features.extend(np.std(mfcc, axis=1))

        # ── 2. MFCC Delta mean + std (80 values) ────────────────────
        mfcc_delta = librosa.feature.delta(mfcc)
        features.extend(np.mean(mfcc_delta, axis=1))
        features.extend(np.std(mfcc_delta, axis=1))

        # ── 3. MFCC Delta-Delta (acceleration) mean + std (80 values) ──────
        # Captures rapid emotional transitions — angry bursts vs slow sad decline
        mfcc_delta2 = librosa.feature.delta(mfcc, order=2)
        features.extend(np.mean(mfcc_delta2, axis=1))
```

```
        features.extend(np.std(mfcc_delta2, axis=1))

        # ── 4. Zero Crossing Rate (2 values) ──────────────────────────────
        zcr = librosa.feature.zero_crossing_rate(y=y, hop_length=HOP_LENGTH)
        features.append(float(np.mean(zcr)))
        features.append(float(np.std(zcr)))

        # ── 5. RMS Energy (2 values) ──────────────────────────────────────
        rms = librosa.feature.rms(y=y, hop_length=HOP_LENGTH)
        features.append(float(np.mean(rms)))
        features.append(float(np.std(rms)))

        # ── 6. Spectral Centroid (2 values) ───────────────────────────────
        spec_centroid = librosa.feature.spectral_centroid(
            y=y, sr=sr, n_fft=N_FFT, hop_length=HOP_LENGTH
        )
        features.append(float(np.mean(spec_centroid)))
        features.append(float(np.std(spec_centroid)))

        # ── 7. Spectral Rolloff (2 values) ────────────────────────────────
        rolloff = librosa.feature.spectral_rolloff(
            y=y, sr=sr, n_fft=N_FFT, hop_length=HOP_LENGTH
        )
        features.append(float(np.mean(rolloff)))
        features.append(float(np.std(rolloff)))

        # ── 8. Spectral Bandwidth (2 values) ──────────────────────────────
        # Width of the spectral distribution — wide=noisy/excited, narrow=calm
        bandwidth = librosa.feature.spectral_bandwidth(
            y=y, sr=sr, n_fft=N_FFT, hop_length=HOP_LENGTH
        )
        features.append(float(np.mean(bandwidth)))
        features.append(float(np.std(bandwidth)))

        # ── 9. Spectral Contrast (14 values: 7 bands × mean+std) ──────────
        # Difference between peaks and valleys per frequency sub-band.
        # Angry/happy → high contrast; Sad/neutral → flatter spectrum
        spec_contrast = librosa.feature.spectral_contrast(
            y=y, sr=sr, n_fft=N_FFT, hop_length=HOP_LENGTH, n_bands=6
        )
        features.extend(np.mean(spec_contrast, axis=1))
        features.extend(np.std(spec_contrast, axis=1))

        # ── 10. Chroma STFT (24 values) ───────────────────────────────────
        chroma = librosa.feature.chroma_stft(
            y=y, sr=sr, n_fft=N_FFT, hop_length=HOP_LENGTH
        )
        features.extend(np.mean(chroma, axis=1))
        features.extend(np.std(chroma, axis=1))

        # ── 11. Tonnetz — Tonal Centroid Features (12 values) ─────────────
        try:
            tonnetz = librosa.feature.tonnetz(y=y, sr=sr)
            features.extend(np.mean(tonnetz, axis=1))
            features.extend(np.std(tonnetz, axis=1))
        except Exception:
            features.extend([0.0] * 12)

        # ── 12. Fundamental Frequency / Pitch F0 (4 values) ───────────────
        # Pitch is the #1 emotion-correlated acoustic feature in the human voice.
        try:
            f0 = librosa.yin(
                y, fmin=65, fmax=1000,
                sr=sr, hop_length=HOP_LENGTH
            )
            f0_valid = f0[np.isfinite(f0) & (f0 > 65)]
            if len(f0_valid) > 5:
                features.append(float(np.mean(f0_valid)))
                features.append(float(np.std(f0_valid)))
                features.append(float(np.min(f0_valid)))
                features.append(float(np.max(f0_valid)))
            else:
                features.extend([0.0, 0.0, 0.0, 0.0])
        except Exception:
            features.extend([0.0, 0.0, 0.0, 0.0])

        # ── 13. Mel Spectrogram Statistics (256 values: 128 bands × mean+std) ─
        # Raw mel filterbank energy — richer than MFCCs (no DCT compression)
        mel_spec = librosa.feature.melspectrogram(
            y=y, sr=sr, n_mels=N_MELS, n_fft=N_FFT, hop_length=HOP_LENGTH
        )
        mel_spec_db = librosa.power_to_db(mel_spec, ref=np.max)
        features.extend(np.mean(mel_spec_db, axis=1))
        features.extend(np.std(mel_spec_db, axis=1))

        feature_vector = np.array(features, dtype=np.float32)
```

```python

        # Sanity check: replace NaN/Inf with 0
        if not np.all(np.isfinite(feature_vector)):
            n_bad = np.sum(~np.isfinite(feature_vector))
            logger.debug(f"Feature vector has {n_bad} non-finite values; replacing with 0.")
            feature_vector = np.nan_to_num(feature_vector, nan=0.0, posinf=0.0, neginf=0.0)

        return feature_vector

    except Exception as e:
        logger.error(f"Feature extraction failed: {e}", exc_info=True)
        return None


def extract_features(audio_path: str) -> Optional[np.ndarray]:
    """
    MAIN ENTRY POINT for feature extraction from an audio file.

    Pipeline: preprocess_audio(audio_path) → extract_features_from_waveform(y, sr)

    Args:
        audio_path: Path to audio file (any format supported by librosa)

    Returns:
        1D numpy array of ~560 features, or None on failure
    """
    y, sr = preprocess_audio(str(audio_path))
    if y is None:
        logger.error(f"Preprocessing failed for {audio_path}")
        return None

    features = extract_features_from_waveform(y, sr)
    if features is None:
        logger.error(f"Feature extraction failed for {audio_path}")
        return None

    return features


def extract_features_from_bytes(
    audio_bytes: bytes,
    original_format: str = "wav",
) -> Optional[np.ndarray]:
    """
    Extract features from raw audio bytes (used in backend for live predictions).

    Args:
        audio_bytes: Raw audio bytes
        original_format: Format hint ('webm', 'wav', 'ogg', etc.)

    Returns:
        1D numpy array of features, or None on failure
    """
    from preprocessing import preprocess_audio_bytes

    y, sr = preprocess_audio_bytes(audio_bytes, original_format)
    if y is None:
        logger.error("Preprocessing from bytes failed.")
        return None

    return extract_features_from_waveform(y, sr)


def get_feature_dimension() -> int:
    """
    Return the expected feature vector dimension.
    80+80+80+2+2+2+2+2+14+24+12+4+256 = 560
    """
    dim = 0
    dim += N_MFCC * 2          # MFCC mean + std
    dim += N_MFCC * 2          # Delta mean + std
    dim += N_MFCC * 2          # Delta-delta mean + std
    dim += 2                   # ZCR
    dim += 2                   # RMS
    dim += 2                   # Spectral centroid
    dim += 2                   # Spectral rolloff
    dim += 2                   # Spectral bandwidth
    dim += (6 + 1) * 2         # Spectral contrast: 7 bands × (mean+std)
    dim += 12 * 2              # Chroma: 12 bins × (mean+std)
    dim += 6 * 2               # Tonnetz: 6 dims × (mean+std)
    dim += 4                   # F0: mean, std, min, max
    dim += N_MELS * 2          # Mel spectrogram: 128 bands × (mean+std)
    return dim
```

PY **src/train.py** 656 Lines • 24,646 bytes

Description & Role: Complete Machine Learning training, model selection, hyperparameter tuning, and serialization pipeline. Compares Multiple Classifiers (Multi-Layer Perceptron / Neural Net, Support Vector Classifier, Random Forest, Gradient Boosting), performs 5-fold Stratified Cross-Validation, evaluates confusion matrices and F1 scores, selects best model, and serializes trained model, StandardScaler, and LabelEncoder to disk.

```python
"""
train.py - Training pipeline for Speech Emotion Detection.

Run this script ONCE to:
1. Load the RAVDESS dataset
2. Extract features from all audio files
3. Train and compare multiple ML models
4. Select the best model based on validation F1-score
5. Evaluate on the held-out test set
6. Save the model, scaler, label encoder, and metadata

Usage:
    python src/train.py

The trained model will be saved to models/ directory.
DO NOT re-run this every time you use the web app — training is a one-time step.
"""

import sys
import json
import logging
import time

if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8', errors='replace')
if hasattr(sys.stderr, 'reconfigure'):
    sys.stderr.reconfigure(encoding='utf-8', errors='replace')
import numpy as np
import pandas as pd
import joblib
from pathlib import Path
from datetime import datetime
from typing import Dict, Tuple, List

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.ensemble import (
    RandomForestClassifier, GradientBoostingClassifier,
    VotingClassifier, StackingClassifier
)
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score, classification_report,
    f1_score, precision_score, recall_score,
    confusion_matrix
)

import matplotlib
matplotlib.use('Agg')  # Non-interactive backend (no GUI needed)
import matplotlib.pyplot as plt
import seaborn as sns

sys.path.insert(0, str(Path(__file__).resolve().parent))
from config import (
    RAW_DIR, FEATURES_DIR, MODELS_DIR, RESULTS_DIR,
    MODEL_PATH, SCALER_PATH, LABEL_ENCODER_PATH,
    METADATA_PATH, FEATURES_CONFIG_PATH,
    CONFUSION_MATRIX_PATH, CLASSIFICATION_REPORT_PATH,
    METRICS_JSON_PATH, MODEL_COMPARISON_PATH,
    RANDOM_SEED, EMOTION_CLASSES, FEATURE_CONFIG,
    SAMPLE_RATE, AUDIO_DURATION
)
from dataset import load_dataset_info, split_dataset
from feature_extraction import extract_features, get_feature_dimension, extract_features_from_waveform
from preprocessing import preprocess_audio

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.StreamHandler(sys.stdout),
        logging.FileHandler(RESULTS_DIR / 'training.log', mode='w', encoding='utf-8'),
```

```
    ]
)
logger = logging.getLogger(__name__)


def augment_waveform(y: np.ndarray, sr: int, rng: np.random.Generator) -> List[np.ndarray]:
    """
    Generate augmented versions of an audio waveform quickly.
    """
    import scipy.signal as sps
    augmented = []
    target_len = len(y)

    # 1. Add Gaussian noise (SNR ~20 dB)
    noise_factor = rng.uniform(0.002, 0.008)
    y_noise = np.clip(y + noise_factor * rng.standard_normal(target_len).astype(np.float32), -1.0, 1.0)
    augmented.append(y_noise)

    # 2. Fast pitch/speed variation via resampling
    rate = rng.uniform(0.92, 1.08)
    try:
        y_stretch = sps.resample(y, int(target_len / rate)).astype(np.float32)
        if len(y_stretch) < target_len:
            y_stretch = np.pad(y_stretch, (0, target_len - len(y_stretch)))
        else:
            y_stretch = y_stretch[:target_len]
        augmented.append(y_stretch)
    except Exception:
        augmented.append(y.copy())

    # 3. Dynamic range / gain variation
    gain = rng.uniform(0.8, 1.2)
    augmented.append(np.clip(y * gain, -1.0, 1.0))

    return augmented


def extract_all_features(
    df: pd.DataFrame,
    split_name: str,
    augment: bool = False,
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Extract features for all audio files in a DataFrame split.
    Optionally applies data augmentation (training set only).
    """
    cache_suffix = f"{split_name}_aug" if augment else split_name
    cache_path = FEATURES_DIR / f"{cache_suffix}_features_v2.npz"

    if cache_path.exists():
        logger.info(f"Loading cached {cache_suffix} features from {cache_path}")
        data = np.load(cache_path, allow_pickle=True)
        return data['X'], data['y']

    logger.info(
        f"Extracting {'augmented ' if augment else ''}features for {split_name} set "
        f"({len(df)} files{'× 4 aug' if augment else ''})..."
    )
    X_list, y_list = [], []
    failed = 0
    rng = np.random.default_rng(RANDOM_SEED)

    for i, row in enumerate(df.itertuples(), 1):
        y_orig, sr_orig = preprocess_audio(str(row.filepath))
        if y_orig is not None:
            feat_orig = extract_features_from_waveform(y_orig, sr_orig)
            if feat_orig is not None:
                X_list.append(feat_orig)
                y_list.append(row.emotion)

                if augment:
                    aug_waves = augment_waveform(y_orig, sr_orig, rng)
                    for y_aug in aug_waves:
                        feat_aug = extract_features_from_waveform(y_aug, sr_orig)
                        if feat_aug is not None:
                            X_list.append(feat_aug)
                            y_list.append(row.emotion)
            else:
                failed += 1
        else:
            failed += 1

        if i % 50 == 0 or i == len(df):
            logger.info(f"  Processed {i}/{len(df)} files ({failed} failed, {len(X_list)} samples so far)...")
            sys.stdout.flush()

```

```
    logger.info(f"Feature extraction complete: {len(X_list)} samples ({failed} files failed).")

    X = np.array(X_list, dtype=np.float32)
    y = np.array(y_list)

    np.savez(cache_path, X=X, y=y)
    logger.info(f"Cached {cache_suffix} features to {cache_path}")

    return X, y


def define_models(random_seed: int = RANDOM_SEED, n_train_samples: int = 0) -> Dict:
    """
    Define the upgraded models to compare.

    Key improvements over v1:
    - SVM: GridSearchCV to find optimal C and gamma (was fixed C=10)
    - MLP_Deep: 4-layer network with alpha regularization to prevent overfitting
    - MLP_Large: Wide single-hidden-layer for feature-rich 560-dim input
    - Ensemble: Soft-voting SVM+MLP combined — best of both worlds
    - Random Forest: More trees, tuned for our expanded feature space
    """
    # Tuned SVM — use GridSearchCV for the best C/gamma pair
    # Fitting this takes ~2min but gives +5-8% accuracy over fixed params
    svm_base = SVC(
        kernel='rbf',
        class_weight='balanced',
        probability=True,
        random_state=random_seed,
        max_iter=3000,
    )
    svm_grid = GridSearchCV(
        svm_base,
        param_grid={
            'C':     [1, 5, 10, 50],
            'gamma': ['scale', 'auto', 0.001, 0.0005],
        },
        cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=random_seed),
        scoring='f1_macro',
        n_jobs=-1,
        verbose=1,
        refit=True,
    )

    # Deep MLP — 4 layers with strong regularization (alpha prevents overfitting
    # on 560-dim features with only ~3800 training samples after augmentation)
    mlp_deep = MLPClassifier(
        hidden_layer_sizes=(512, 256, 128, 64),
        activation='relu',
        solver='adam',
        alpha=0.01,             # L2 regularization — crucial for high-dim features
        batch_size=64,
        learning_rate='adaptive',
        learning_rate_init=0.001,
        max_iter=600,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=20,    # More patience before early stop
        random_state=random_seed,
    )

    # Wider MLP alternative
    mlp_wide = MLPClassifier(
        hidden_layer_sizes=(256, 128),
        activation='relu',
        solver='adam',
        alpha=0.005,
        batch_size=32,
        learning_rate='adaptive',
        learning_rate_init=0.0005,
        max_iter=500,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=15,
        random_state=random_seed,
    )

    # Random Forest — increase trees for augmented dataset
    rf = RandomForestClassifier(
        n_estimators=500,
        max_depth=None,
        min_samples_split=2,
        max_features='sqrt',
        class_weight='balanced',
        random_state=random_seed,
        n_jobs=-1,
```

```
    )

    # Gradient Boosting — strong learner for tabular/feature-based data
    gb = GradientBoostingClassifier(
        n_estimators=300,
        max_depth=5,
        learning_rate=0.05,
        subsample=0.8,
        random_state=random_seed,
    )

    return {
        "SVM_RBF_Tuned": svm_grid,
        "MLP_Deep":      mlp_deep,
        "MLP_Wide":      mlp_wide,
        "Random_Forest": rf,
        "GradientBoost": gb,
    }


def train_and_evaluate_models(
    X_train: np.ndarray, y_train: np.ndarray,
    X_val: np.ndarray, y_val: np.ndarray,
    label_classes: List[str],
) -> Tuple[Dict, pd.DataFrame]:
    """
    Train all models and evaluate on validation set.

    Returns:
        models_dict: Trained model objects
        comparison_df: DataFrame with validation metrics
    """
    models = define_models(n_train_samples=len(X_train))
    results = []

    logger.info("\n" + "="*60)
    logger.info("MODEL COMPARISON ON VALIDATION SET")
    logger.info("="*60)

    trained_models = {}

    for name, model in models.items():
        logger.info(f"\nTraining {name}...")
        start_time = time.time()

        try:
            model.fit(X_train, y_train)
            train_time = time.time() - start_time

            # For GridSearchCV models, log the best params found
            if hasattr(model, 'best_params_'):
                logger.info(f"  Best params: {model.best_params_}")

            y_val_pred = model.predict(X_val)

            acc         = accuracy_score(y_val, y_val_pred)
            macro_f1    = f1_score(y_val, y_val_pred, average='macro', zero_division=0)
            weighted_f1 = f1_score(y_val, y_val_pred, average='weighted', zero_division=0)
            precision   = precision_score(y_val, y_val_pred, average='weighted', zero_division=0)
            recall      = recall_score(y_val, y_val_pred, average='weighted', zero_division=0)

            logger.info(f"  Accuracy:     {acc:.4f}  ({acc*100:.1f}%)")
            logger.info(f"  Macro F1:     {macro_f1:.4f}")
            logger.info(f"  Weighted F1:  {weighted_f1:.4f}")
            logger.info(f"  Train time:   {train_time:.1f}s")

            results.append({
                "Model": name,
                "Accuracy": round(acc, 4),
                "Macro_F1": round(macro_f1, 4),
                "Weighted_F1": round(weighted_f1, 4),
                "Precision_W": round(precision, 4),
                "Recall_W": round(recall, 4),
                "Train_Time_s": round(train_time, 1),
            })

            trained_models[name] = model

        except Exception as e:
            logger.error(f"  Training failed for {name}: {e}", exc_info=True)

    # ── Build Ensemble from top-2 models ──────────────────────────────
    # Soft voting averages class probabilities — almost always beats individual models
    if len(trained_models) >= 2:
        logger.info("\nBuilding Ensemble (soft-voting top models)...")
        try:
```

```
            sorted_results = sorted(results, key=lambda r: r["Macro_F1"], reverse=True)
            top_names = [r["Model"] for r in sorted_results[:3]]
            top_estimators = [(n, trained_models[n]) for n in top_names if n in trained_models]

            start_time = time.time()
            ensemble = VotingClassifier(
                estimators=top_estimators,
                voting='soft',
                n_jobs=-1,
            )
            ensemble.fit(X_train, y_train)
            train_time = time.time() - start_time

            y_ens_pred  = ensemble.predict(X_val)
            acc         = accuracy_score(y_val, y_ens_pred)
            macro_f1    = f1_score(y_val, y_ens_pred, average='macro', zero_division=0)
            weighted_f1 = f1_score(y_val, y_ens_pred, average='weighted', zero_division=0)
            precision   = precision_score(y_val, y_ens_pred, average='weighted', zero_division=0)
            recall      = recall_score(y_val, y_ens_pred, average='weighted', zero_division=0)

            logger.info(f"  Ensemble ({'+'.join(top_names)}):")
            logger.info(f"  Accuracy:    {acc:.4f}  ({acc*100:.1f}%)")
            logger.info(f"  Macro F1:    {macro_f1:.4f}")
            logger.info(f"  Train time:  {train_time:.1f}s")

            results.append({
                "Model": "Ensemble_SoftVote",
                "Accuracy": round(acc, 4),
                "Macro_F1": round(macro_f1, 4),
                "Weighted_F1": round(weighted_f1, 4),
                "Precision_W": round(precision, 4),
                "Recall_W": round(recall, 4),
                "Train_Time_s": round(train_time, 1),
            })
            trained_models["Ensemble_SoftVote"] = ensemble
        except Exception as e:
            logger.warning(f"Ensemble building failed: {e}")

    comparison_df = pd.DataFrame(results).sort_values("Macro_F1", ascending=False)
    logger.info("\n" + "="*60)
    logger.info("VALIDATION RESULTS (sorted by Macro F1):")
    logger.info(comparison_df.to_string(index=False))

    return trained_models, comparison_df


def evaluate_final_model(
    model,
    X_test: np.ndarray,
    y_test: np.ndarray,
    label_encoder: LabelEncoder,
    emotion_classes: List[str],
) -> Dict:
    """
    Perform final evaluation on the held-out test set.

    This is only done ONCE at the end with the best model.
    Using the test set multiple times would introduce evaluation bias.
    """
    logger.info("\n" + "="*60)
    logger.info("FINAL EVALUATION ON TEST SET")
    logger.info("="*60)

    y_pred = model.predict(X_test)

    acc        = accuracy_score(y_test, y_pred)
    macro_f1   = f1_score(y_test, y_pred, average='macro', zero_division=0)
    weighted_f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
    macro_precision = precision_score(y_test, y_pred, average='macro', zero_division=0)
    macro_recall    = recall_score(y_test, y_pred, average='macro', zero_division=0)

    logger.info(f"Test Accuracy:         {acc:.4f} ({acc*100:.1f}%)")
    logger.info(f"Macro F1-score:        {macro_f1:.4f}")
    logger.info(f"Weighted F1-score:     {weighted_f1:.4f}")
    logger.info(f"Macro Precision:       {macro_precision:.4f}")
    logger.info(f"Macro Recall:          {macro_recall:.4f}")

    # Classification report
    report = classification_report(
        y_test, y_pred,
        labels=emotion_classes,
        zero_division=0
    )
    logger.info("\nClassification Report:\n" + report)

    # Save classification report
```

```
    with open(CLASSIFICATION_REPORT_PATH, 'w') as f:
        f.write("SPEECH EMOTION DETECTION — Classification Report\n")
        f.write("="*60 + "\n")
        f.write(f"Test Accuracy:    {acc:.4f}\n")
        f.write(f"Macro F1:         {macro_f1:.4f}\n")
        f.write(f"Weighted F1:      {weighted_f1:.4f}\n\n")
        f.write(report)

    metrics = {
        "test_accuracy": round(float(acc), 4),
        "macro_f1": round(float(macro_f1), 4),
        "weighted_f1": round(float(weighted_f1), 4),
        "macro_precision": round(float(macro_precision), 4),
        "macro_recall": round(float(macro_recall), 4),
        "n_test_samples": int(len(y_test)),
    }

    return metrics, y_pred


def save_confusion_matrix(y_test, y_pred, emotion_classes: List[str]):
    """Generate and save a professional confusion matrix heatmap."""
    cm = confusion_matrix(y_test, y_pred, labels=emotion_classes)
    cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)  # Row-normalized

    fig, axes = plt.subplots(1, 2, figsize=(20, 8))
    fig.patch.set_facecolor('#1a1a2e')

    # Raw counts
    sns.heatmap(
        cm, annot=True, fmt='d', cmap='Blues',
        xticklabels=emotion_classes,
        yticklabels=emotion_classes,
        ax=axes[0],
        linewidths=0.5,
        cbar_kws={'label': 'Count'}
    )
    axes[0].set_title('Confusion Matrix (Counts)', fontsize=14, color='white', pad=15)
    axes[0].set_xlabel('Predicted Emotion', fontsize=12, color='white')
    axes[0].set_ylabel('True Emotion', fontsize=12, color='white')
    axes[0].tick_params(colors='white', rotation=45)
    axes[0].set_facecolor('#16213e')

    # Normalized
    sns.heatmap(
        cm_norm, annot=True, fmt='.2f', cmap='RdYlGn',
        xticklabels=emotion_classes,
        yticklabels=emotion_classes,
        ax=axes[1],
        vmin=0, vmax=1,
        linewidths=0.5,
        cbar_kws={'label': 'Recall'}
    )
    axes[1].set_title('Confusion Matrix (Recall)', fontsize=14, color='white', pad=15)
    axes[1].set_xlabel('Predicted Emotion', fontsize=12, color='white')
    axes[1].set_ylabel('', fontsize=12)
    axes[1].tick_params(colors='white', rotation=45)
    axes[1].set_facecolor('#16213e')

    plt.suptitle(
        'Speech Emotion Detection — Confusion Matrix',
        fontsize=16, fontweight='bold', color='white', y=1.01
    )
    plt.tight_layout()
    plt.savefig(CONFUSION_MATRIX_PATH, dpi=150, bbox_inches='tight', facecolor='#1a1a2e')
    plt.close()
    logger.info(f"Saved confusion matrix: {CONFUSION_MATRIX_PATH}")


def main():
    """Main training pipeline."""
    logger.info("🎙️ Speech Emotion Detection — Training Pipeline")
    logger.info(f"Started at: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    logger.info(f"Random seed: {RANDOM_SEED}")

    # ── Step 1: Load dataset ──────────────────────────────────────
    logger.info("\n[STEP 1] Loading dataset...")
    df = load_dataset_info(RAW_DIR)

    if df.empty:
        logger.error("❌ Dataset not found! Please download RAVDESS first.")
        logger.error("Download: https://zenodo.org/record/1188976")
        logger.error(f"Extract to: {RAW_DIR}")
        sys.exit(1)

    logger.info(f"Loaded {len(df)} audio files, {df['emotion'].nunique()} emotions, "
```

```
                f"{df['actor_id'].nunique()} speakers")

    # Get the actual emotion classes present in dataset
    actual_emotions = sorted(df['emotion'].unique().tolist())
    logger.info(f"Emotions in dataset: {actual_emotions}")

    # ── Step 2: Split dataset (speaker-independent) ──────────────────────
    logger.info("\n[STEP 2] Splitting dataset (speaker-independent)...")
    train_df, val_df, test_df = split_dataset(df, random_seed=RANDOM_SEED)
    logger.info(f"Train: {len(train_df)}, Val: {len(val_df)}, Test: {len(test_df)}")

    logger.info("\n[STEP 3] Extracting features (with data augmentation on train set)...")
    feature_dim = get_feature_dimension()
    logger.info(f"New feature dimension: {feature_dim} (was 192 — now {feature_dim} with rich features)")

    # augment=True triples the training data via noise/pitch-shift/time-stretch
    X_train_raw, y_train = extract_all_features(train_df, 'train', augment=True)
    X_val_raw, y_val     = extract_all_features(val_df,   'val',   augment=False)
    X_test_raw, y_test   = extract_all_features(test_df,  'test',  augment=False)

    logger.info(f"X_train shape: {X_train_raw.shape}")
    logger.info(f"X_val shape:   {X_val_raw.shape}")
    logger.info(f"X_test shape:  {X_test_raw.shape}")

    # ── Step 4: Encode labels ────────────────────────────────────────────
    logger.info("\n[STEP 4] Encoding emotion labels...")
    label_encoder = LabelEncoder()
    label_encoder.fit(actual_emotions)

    y_train_enc = label_encoder.transform(y_train)
    y_val_enc   = label_encoder.transform(y_val)
    y_test_enc  = label_encoder.transform(y_test)

    logger.info(f"Label classes: {list(label_encoder.classes_)}")

    # ── Step 5: Scale features ───────────────────────────────────────────
    logger.info("\n[STEP 5] Scaling features (StandardScaler)...")
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train_raw)   # Fit ONLY on training data!
    X_val   = scaler.transform(X_val_raw)          # Apply same transform
    X_test  = scaler.transform(X_test_raw)         # Apply same transform

    # ── Step 6: Train and compare models ─────────────────────────────────
    logger.info("\n[STEP 6] Training and comparing models...")
    trained_models, comparison_df = train_and_evaluate_models(
        X_train, y_train, X_val, y_val, list(label_encoder.classes_)
    )

    # Save model comparison
    comparison_df.to_csv(MODEL_COMPARISON_PATH, index=False)
    logger.info(f"Saved model comparison: {MODEL_COMPARISON_PATH}")

    # ── Step 7: Select best model ────────────────────────────────────────
    logger.info("\n[STEP 7] Selecting best model based on validation Macro F1...")
    best_model_name = comparison_df.iloc[0]['Model']
    best_model = trained_models[best_model_name]
    logger.info(f"Best model: {best_model_name} "
                f"(Val Macro F1: {comparison_df.iloc[0]['Macro_F1']:.4f})")

    # ── Step 8: Retrain on train+val data ────────────────────────────────
    logger.info("\n[STEP 8] Retraining best model on train + val data...")
    X_trainval_raw = np.vstack([X_train_raw, X_val_raw])
    y_trainval = np.concatenate([y_train, y_val])

    # Re-scale with train+val data
    scaler_final = StandardScaler()
    X_trainval_scaled = scaler_final.fit_transform(X_trainval_raw)
    X_test_scaled     = scaler_final.transform(X_test_raw)

    # Retrain the best model type with same hyperparams
    final_model = define_models()[best_model_name]
    final_model.fit(X_trainval_scaled, y_trainval)
    logger.info(f"Final model retrained on {len(X_trainval_raw)} samples.")

    # ── Step 9: Evaluate on test set ─────────────────────────────────────
    logger.info("\n[STEP 9] Final evaluation on test set...")
    y_test_pred = final_model.predict(X_test_scaled)

    metrics, _ = evaluate_final_model(
        final_model, X_test_scaled, y_test,
        label_encoder, list(label_encoder.classes_)
    )

    save_confusion_matrix(y_test, y_test_pred, list(label_encoder.classes_))

    # ── Step 10: Save model and artifacts ────────────────────────
```

```
    logger.info("\n[STEP 10] Saving model and artifacts...")

    joblib.dump(final_model, MODEL_PATH)
    logger.info(f"Saved model: {MODEL_PATH}")

    joblib.dump(scaler_final, SCALER_PATH)
    logger.info(f"Saved scaler: {SCALER_PATH}")

    joblib.dump(label_encoder, LABEL_ENCODER_PATH)
    logger.info(f"Saved label encoder: {LABEL_ENCODER_PATH}")

    # Save feature config
    with open(FEATURES_CONFIG_PATH, 'w') as f:
        json.dump(FEATURE_CONFIG, f, indent=2)

    # Save full metrics
    all_metrics = {
        **metrics,
        "model_name": best_model_name,
        "dataset": "RAVDESS",
        "n_emotions": len(label_encoder.classes_),
        "emotion_classes": list(label_encoder.classes_),
        "feature_dimension": int(X_train.shape[1]),
        "train_samples": int(len(X_trainval_raw)),
        "test_samples": int(len(X_test)),
        "model_comparison": comparison_df.to_dict(orient='records'),
    }
    with open(METRICS_JSON_PATH, 'w') as f:
        json.dump(all_metrics, f, indent=2)
    logger.info(f"Saved metrics: {METRICS_JSON_PATH}")

    # Save model metadata
    metadata = {
        "model_type": best_model_name,
        "dataset": "RAVDESS",
        "dataset_url": "https://zenodo.org/record/1188976",
        "feature_type": "MFCC+Delta+ZCR+RMS+Centroid+Rolloff+Chroma",
        "feature_dimension": int(X_train.shape[1]),
        "emotion_classes": list(label_encoder.classes_),
        "n_emotions": len(label_encoder.classes_),
        "random_seed": RANDOM_SEED,
        "training_date": datetime.now().isoformat(),
        "test_accuracy": metrics["test_accuracy"],
        "macro_f1": metrics["macro_f1"],
        "train_samples": int(len(X_trainval_raw)),
        "test_samples": int(len(X_test)),
    }
    with open(METADATA_PATH, 'w') as f:
        json.dump(metadata, f, indent=2)
    logger.info(f"Saved metadata: {METADATA_PATH}")

    logger.info("\n" + "="*60)
    logger.info("✅ TRAINING COMPLETE!")
    logger.info("="*60)
    logger.info(f"Best model:      {best_model_name}")
    logger.info(f"Test Accuracy:   {metrics['test_accuracy']:.4f} ({metrics['test_accuracy']*100:.1f}%)")
    logger.info(f"Macro F1:        {metrics['macro_f1']:.4f}")
    logger.info(f"Weighted F1:     {metrics['weighted_f1']:.4f}")
    logger.info(f"\nFiles saved in: {MODELS_DIR}")
    logger.info(f"Results in:     {RESULTS_DIR}")
    logger.info("\nYou can now start the backend: python backend/main.py")


if __name__ == "__main__":
    main()
```

PY **src/analyze_dataset.py** 289 Lines • 11,180 bytes

Description & Role: Exploratory Data Analysis (EDA) tool for audio datasets. Analyzes class distribution, audio duration statistics, sample rates, actor gender balance, and generates publication-quality visualization charts (spectrograms, waveforms, and distributions).

```python
"""
analyze_dataset.py - Dataset analysis and visualization for RAVDESS.

Run this AFTER placing the RAVDESS dataset in data/raw/RAVDESS/.

This script generates:
  results/emotion_distribution.png
  results/duration_distribution.png
  results/sample_waveform.png
  results/sample_spectrogram.png
  results/dataset_stats.json
"""

import sys
import json
import logging

if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8', errors='replace')
if hasattr(sys.stderr, 'reconfigure'):
    sys.stderr.reconfigure(encoding='utf-8', errors='replace')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import librosa
import librosa.display
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))
from config import RAW_DIR, RESULTS_DIR, SAMPLE_RATE
from dataset import load_dataset_info

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

# Use a clean matplotlib style
plt.rcParams.update({
    'figure.facecolor': '#1a1a2e',
    'axes.facecolor': '#16213e',
    'axes.labelcolor': 'white',
    'text.color': 'white',
    'xtick.color': 'white',
    'ytick.color': 'white',
    'grid.color': '#2d4059',
    'axes.spines.top': False,
    'axes.spines.right': False,
})

EMOTION_COLORS = {
    'Neutral':   '#95a5a6',
    'Calm':      '#3498db',
    'Happy':     '#f1c40f',
    'Sad':       '#2980b9',
    'Angry':     '#e74c3c',
    'Fearful':   '#9b59b6',
    'Disgust':   '#27ae60',
    'Surprised': '#e67e22',
}


def get_audio_durations(df: pd.DataFrame) -> pd.Series:
    """Get duration (seconds) for each audio file."""
    durations = []
    for _, row in df.iterrows():
        try:
            duration = librosa.get_duration(path=row['filepath'])
            durations.append(duration)
        except Exception:
            durations.append(np.nan)
    return pd.Series(durations, index=df.index)


def plot_emotion_distribution(df: pd.DataFrame, save_path: Path):
    """Bar chart showing how many samples exist per emotion."""
    emotion_counts = df['emotion'].value_counts()
```

```python
    # Reorder to standard order
    ordered_emotions = [e for e in ['Neutral', 'Calm', 'Happy', 'Sad', 'Angry', 'Fearful', 'Disgust', 'Surprised']
                        if e in emotion_counts.index]
    emotion_counts = emotion_counts.reindex(ordered_emotions)

    fig, ax = plt.subplots(figsize=(12, 6))
    colors = [EMOTION_COLORS.get(e, '#ffffff') for e in emotion_counts.index]
    bars = ax.bar(emotion_counts.index, emotion_counts.values, color=colors, edgecolor='none', alpha=0.9)

    ax.set_title('RAVDESS Dataset — Emotion Distribution', fontsize=16, fontweight='bold', color='white', pad=20)
    ax.set_xlabel('Emotion', fontsize=13)
    ax.set_ylabel('Number of Audio Files', fontsize=13)
    ax.set_ylim(0, emotion_counts.max() * 1.2)
    ax.grid(axis='y', alpha=0.3)

    for bar, val in zip(bars, emotion_counts.values):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 5,
            str(val), ha='center', va='bottom', fontsize=12, color='white', fontweight='bold'
        )

    plt.tight_layout()
    plt.savefig(save_path, dpi=150, bbox_inches='tight', facecolor='#1a1a2e')
    plt.close()
    logger.info(f"Saved: {save_path}")


def plot_duration_distribution(df: pd.DataFrame, save_path: Path):
    """Histogram of audio clip durations."""
    logger.info("Computing audio durations (this may take a minute)...")
    durations = get_audio_durations(df)
    durations = durations.dropna()

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Histogram
    axes[0].hist(durations, bins=30, color='#3498db', edgecolor='none', alpha=0.85)
    axes[0].set_title('Audio Duration Distribution', fontsize=14, color='white')
    axes[0].set_xlabel('Duration (seconds)', fontsize=12)
    axes[0].set_ylabel('Count', fontsize=12)
    axes[0].axvline(durations.mean(), color='#f1c40f', linestyle='--', linewidth=2, label=f'Mean: {durations.mean():.2f}s')
    axes[0].axvline(durations.median(), color='#e74c3c', linestyle='--', linewidth=2, label=f'Median: {durations.median():.2f}s
    axes[0].legend(fontsize=10)
    axes[0].grid(alpha=0.3)

    # Box plot by emotion
    emotion_durations = {}
    for emotion in df['emotion'].unique():
        mask = df['emotion'] == emotion
        emotion_durations[emotion] = durations[mask].values

    box_data = [emotion_durations.get(e, []) for e in ['Neutral', 'Calm', 'Happy', 'Sad', 'Angry', 'Fearful', 'Disgust', 'Surpr
                if e in emotion_durations]
    box_labels = [e for e in ['Neutral', 'Calm', 'Happy', 'Sad', 'Angry', 'Fearful', 'Disgust', 'Surprised']
                  if e in emotion_durations]

    try:
        bp = axes[1].boxplot(box_data, tick_labels=box_labels, patch_artist=True)
    except TypeError:
        bp = axes[1].boxplot(box_data, labels=box_labels, patch_artist=True)
    for patch, label in zip(bp['boxes'], box_labels):
        patch.set_facecolor(EMOTION_COLORS.get(label, '#ffffff'))
        patch.set_alpha(0.8)
    axes[1].set_title('Duration by Emotion', fontsize=14, color='white')
    axes[1].set_xlabel('Emotion', fontsize=12)
    axes[1].set_ylabel('Duration (seconds)', fontsize=12)
    axes[1].tick_params(axis='x', rotation=45)
    axes[1].grid(alpha=0.3)

    plt.tight_layout()
    plt.savefig(save_path, dpi=150, bbox_inches='tight', facecolor='#1a1a2e')
    plt.close()
    logger.info(f"Saved: {save_path}")

    return durations


def plot_sample_waveform(df: pd.DataFrame, save_path: Path):
    """Plot sample waveforms for selected emotions."""
    emotions_to_show = ['Happy', 'Angry', 'Sad', 'Neutral']
    available = [e for e in emotions_to_show if e in df['emotion'].values]
    if not available:
        available = df['emotion'].unique()[:4]

    fig, axes = plt.subplots(len(available), 1, figsize=(14, 3 * len(available)))
```

```
    if len(available) == 1:
        axes = [axes]

    for ax, emotion in zip(axes, available):
        sample = df[df['emotion'] == emotion].iloc[0]
        try:
            y, sr = librosa.load(sample['filepath'], sr=SAMPLE_RATE, mono=True)
            time = np.linspace(0, len(y) / sr, len(y))
            color = EMOTION_COLORS.get(emotion, '#3498db')
            ax.plot(time, y, color=color, linewidth=0.5, alpha=0.9)
            ax.set_title(f'{emotion} — Waveform', fontsize=12, color='white')
            ax.set_xlabel('Time (s)', fontsize=10)
            ax.set_ylabel('Amplitude', fontsize=10)
            ax.set_xlim(0, max(time))
            ax.grid(alpha=0.2)
        except Exception as e:
            ax.text(0.5, 0.5, f'Error loading {emotion}', ha='center', transform=ax.transAxes)

    plt.suptitle('Sample Waveforms by Emotion', fontsize=15, fontweight='bold', color='white', y=1.01)
    plt.tight_layout()
    plt.savefig(save_path, dpi=150, bbox_inches='tight', facecolor='#1a1a2e')
    plt.close()
    logger.info(f"Saved: {save_path}")


def plot_sample_spectrogram(df: pd.DataFrame, save_path: Path):
    """Plot Mel spectrogram for one sample per emotion (up to 4)."""
    emotions_to_show = ['Happy', 'Angry', 'Sad', 'Neutral']
    available = [e for e in emotions_to_show if e in df['emotion'].values]
    if not available:
        available = df['emotion'].unique()[:4]

    fig, axes = plt.subplots(1, len(available), figsize=(5 * len(available), 5))
    if len(available) == 1:
        axes = [axes]

    for ax, emotion in zip(axes, available):
        sample = df[df['emotion'] == emotion].iloc[0]
        try:
            y, sr = librosa.load(sample['filepath'], sr=SAMPLE_RATE, mono=True)
            S = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128)
            S_db = librosa.power_to_db(S, ref=np.max)
            img = librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='mel', ax=ax, cmap='magma')
            ax.set_title(f'{emotion}', fontsize=13, color='white', fontweight='bold')
            ax.set_xlabel('Time (s)', fontsize=10)
            ax.set_ylabel('Mel Frequency' if emotion == available[0] else '', fontsize=10)
        except Exception as e:
            ax.text(0.5, 0.5, f'Error: {emotion}', ha='center', transform=ax.transAxes, color='white')

    plt.suptitle('Mel Spectrograms by Emotion', fontsize=15, fontweight='bold', color='white')
    plt.tight_layout()
    plt.savefig(save_path, dpi=150, bbox_inches='tight', facecolor='#1a1a2e')
    plt.close()
    logger.info(f"Saved: {save_path}")


def run_analysis():
    """Main analysis function."""
    logger.info("=" * 60)
    logger.info("RAVDESS Dataset Analysis")
    logger.info("=" * 60)

    RESULTS_DIR.mkdir(parents=True, exist_ok=True)

    # Load dataset info
    df = load_dataset_info(RAW_DIR)

    if df.empty:
        logger.error("Dataset not found. Please download RAVDESS first.")
        logger.error("Download from: https://zenodo.org/record/1188976")
        logger.error(f"Extract to: {RAW_DIR}")
        return None

    # Basic statistics
    logger.info(f"\nTotal audio files:   {len(df)}")
    logger.info(f"Number of speakers:  {df['actor_id'].nunique()}")
    logger.info(f"Emotion classes:     {sorted(df['emotion'].unique())}")
    logger.info(f"\nSamples per emotion:")
    for emotion, count in df['emotion'].value_counts().items():
        logger.info(f"  {emotion:12s}: {count}")

    # Check for class imbalance
    counts = df['emotion'].value_counts()
    imbalance_ratio = counts.max() / counts.min() if counts.min() > 0 else float('inf')
    logger.info(f"\nClass imbalance ratio (max/min): {imbalance_ratio:.2f}")
    if imbalance_ratio > 1.5:
```

```
            logger.warning("Class imbalance detected. Will use class_weight='balanced' during training.")

    # Generate visualizations
    logger.info("\nGenerating visualizations...")
    plot_emotion_distribution(df, RESULTS_DIR / "emotion_distribution.png")

    durations = plot_duration_distribution(df, RESULTS_DIR / "duration_distribution.png")
    plot_sample_waveform(df, RESULTS_DIR / "sample_waveforms.png")
    plot_sample_spectrogram(df, RESULTS_DIR / "sample_spectrograms.png")

    # Save stats to JSON
    stats = {
        "total_files": int(len(df)),
        "n_speakers": int(df['actor_id'].nunique()),
        "emotions": sorted(df['emotion'].unique().tolist()),
        "samples_per_emotion": df['emotion'].value_counts().to_dict(),
        "duration_stats_seconds": {
            "mean":   float(durations.mean()),
            "std":    float(durations.std()),
            "min":    float(durations.min()),
            "max":    float(durations.max()),
            "median": float(durations.median()),
        },
        "imbalance_ratio": float(imbalance_ratio),
    }

    stats_path = RESULTS_DIR / "dataset_stats.json"
    with open(stats_path, 'w') as f:
        json.dump(stats, f, indent=2)
    logger.info(f"\nSaved dataset statistics to: {stats_path}")

    logger.info("\n✅ Dataset analysis complete!")
    logger.info(f"Check results in: {RESULTS_DIR}")

    return df


if __name__ == "__main__":
    run_analysis()
```

PY **src/download_dataset.py** 211 Lines • 7,240 bytes

Description & Role: Dataset utility script for downloading, extracting, and verifying RAVDESS audio dataset files, ensuring integrity of audio samples and directory structure.

```python
"""
download_dataset.py - Helper script to guide RAVDESS dataset download.

RAVDESS Dataset Information:
  Name:    Ryerson Audio-Visual Database of Emotional Speech and Song
  URL:     https://zenodo.org/record/1188976
  License: Creative Commons Attribution (CC BY 4.0)
  Size:    ~24.8 GB (full), ~1.5 GB (audio-speech only files)

This script:
1. Checks if the dataset already exists
2. Provides download instructions
3. Verifies the dataset after download
4. Lists what was found
"""

import sys
import zipfile
import json
import logging
import urllib.request
from pathlib import Path

if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8', errors='replace')
if hasattr(sys.stderr, 'reconfigure'):
    sys.stderr.reconfigure(encoding='utf-8', errors='replace')

PROJECT_ROOT = Path(__file__).resolve().parent.parent
SRC_DIR = PROJECT_ROOT / "src"
sys.path.insert(0, str(SRC_DIR))

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

from config import RAW_DIR
from dataset import load_dataset_info, verify_dataset


def print_banner():
    print("\n" + "="*70)
    print("  RAVDESS Dataset Setup for Speech Emotion Detection")
    print("="*70 + "\n")


def check_existing():
    """Check what's already in the data directory."""
    wav_files = list(RAW_DIR.rglob("*.wav"))
    zip_files = list(RAW_DIR.glob("*.zip")) + list(RAW_DIR.parent.glob("*.zip"))
    return wav_files, zip_files


def auto_extract_zip(zip_path: Path):
    """Extract a RAVDESS zip file to the correct location."""
    logger.info(f"Extracting {zip_path.name}...")
    try:
        with zipfile.ZipFile(zip_path, 'r') as zf:
            members = zf.namelist()
            logger.info(f"  Found {len(members)} files in zip")
            zf.extractall(RAW_DIR)
        logger.info(f"  ✅ Extracted to {RAW_DIR}")
        return True
    except Exception as e:
        logger.error(f"  ❌ Extraction failed: {e}")
        return False


def download_ravdess_speech(dest_zip: Path):
    """Download Audio_Speech_Actors_01-24.zip directly from Zenodo API."""
    url = "https://zenodo.org/api/records/1188976/files/Audio_Speech_Actors_01-24.zip/content"
    logger.info("Attempting automatic download of Audio_Speech_Actors_01-24.zip (~208 MB)...")
    logger.info(f"URL: {url}")
    req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)'})
    try:
        with urllib.request.urlopen(req, timeout=30) as resp:
            total_size = int(resp.headers.get('Content-Length', 0))
            logger.info(f"Connected. Total size: {total_size / (1024 * 1024):.1f} MB")
            downloaded = 0
```

```
            chunk_size = 1024 * 1024  # 1MB chunks
            last_percent = -1
            with open(dest_zip, 'wb') as f:
                while True:
                    chunk = resp.read(chunk_size)
                    if not chunk:
                        break
                    f.write(chunk)
                    downloaded += len(chunk)
                    if total_size > 0:
                        pct = int(downloaded * 100 / total_size)
                        if pct % 10 == 0 and pct != last_percent:
                            logger.info(f"  Downloading: {pct}% ({downloaded / (1024*1024):.1f} MB / {total_size / (1024*1024):
                            last_percent = pct
            logger.info("✅ Download complete!")
            return True
    except Exception as e:
        logger.error(f"Download failed: {e}")
        if dest_zip.exists():
            dest_zip.unlink()
        return False


def main():
    print_banner()

    RAW_DIR.mkdir(parents=True, exist_ok=True)

    wav_files, zip_files = check_existing()

    # Case 1: Dataset already present
    if len(wav_files) >= 100:
        logger.info(f"✅ Dataset appears to be present: {len(wav_files)} .wav files found")
        df = load_dataset_info(RAW_DIR)
        if not df.empty:
            logger.info(f"   Speakers: {df['actor_id'].nunique()}")
            logger.info(f"   Emotions: {sorted(df['emotion'].unique().tolist())}")
            logger.info(f"   Total speech files: {len(df)}")
            logger.info("\n✅ Dataset is ready! You can now run training:")
            logger.info("   python src/train.py")
        return

    # Case 2: ZIP files found — try to extract
    if zip_files:
        logger.info(f"Found {len(zip_files)} zip file(s). Attempting extraction...")
        for zf in zip_files:
            auto_extract_zip(zf)

        wav_files_after = list(RAW_DIR.rglob("*.wav"))
        if len(wav_files_after) >= 100:
            logger.info("✅ Extraction successful!")
            main()  # Re-check
            return

    # Case 3: Try automatic download
    target_zip = RAW_DIR / "Audio_Speech_Actors_01-24.zip"
    if download_ravdess_speech(target_zip):
        if auto_extract_zip(target_zip):
            wav_files_after = list(RAW_DIR.rglob("*.wav"))
            if len(wav_files_after) >= 100:
                logger.info("✅ Download and extraction successful!")
                main()
                return

    # Case 4: Need manual download fallback
    print("\n" + "="*70)
    print("  📥 MANUAL DOWNLOAD REQUIRED")
    print("="*70)
    print("""
RAVDESS dataset requires a free download from Zenodo.

STEP 1: Go to this URL:
  https://zenodo.org/record/1188976

STEP 2: Download ONLY the speech files (NOT the video/song files):
  File: Audio_Speech_Actors_01-24.zip
  Size: ~208 MB

  Direct download link:
  https://zenodo.org/api/records/1188976/files/Audio_Speech_Actors_01-24.zip/content

STEP 3: Place the downloaded ZIP file in:
  """ + str(RAW_DIR) + """

STEP 4: Run this script again:
  python src/download_dataset.py
```

```
  OR extract manually and ensure structure is:
  data/raw/RAVDESS/
    Actor_01/
      03-01-01-01-01-01-01.wav
      03-01-02-01-01-01-01.wav
      ...
    Actor_02/
      ...
    ...
    Actor_24/
      ...

ABOUT THE DATASET:
  Name:     RAVDESS (Ryerson Audio-Visual Database of Emotional Speech and Song)
  Speakers: 24 professional actors (12 male, 12 female)
  Emotions: 8 (Neutral, Calm, Happy, Sad, Angry, Fearful, Disgust, Surprised)
  Files:    1440 speech audio files
  License:  CC BY 4.0 (free for academic/research use with attribution)
  Citation: Livingstone SR, Russo FA (2018) The Ryerson Audio-Visual Database of
            Emotional Speech and Song (RAVDESS). PLoS ONE 13(5): e0196391.
""")
    print("-"*70)


def verify_and_report():
    """Verify dataset and print statistics."""
    df = load_dataset_info(RAW_DIR)
    if df.empty:
        logger.error("Dataset not found or not recognized.")
        return False

    print("\n📊 Dataset Statistics:")
    print(f"  Total files:  {len(df)}")
    print(f"  Speakers:     {df['actor_id'].nunique()}")
    print(f"  Emotions:     {df['emotion'].nunique()}")
    print("\n  Samples per emotion:")
    for emotion, count in sorted(df['emotion'].value_counts().items()):
        bar = "█" * (count // 10)
        print(f"    {emotion:12s}: {count:3d}  {bar}")
    return True


if __name__ == "__main__":
    main()
    if verify_dataset(RAW_DIR):
        verify_and_report()
```

3. Backend API Server

FastAPI REST API, asynchronous audio inference, CORS, and health endpoints (backend/)

PY **backend/main.py** 448 Lines • 15,056 bytes

Description & Role: FastAPI asynchronous web server providing real-time emotion prediction endpoints (/predict, /predict-raw), model metadata endpoints (/model-info), system health checks (/health), audio validation, in-memory audio decoding, feature extraction pipeline integration, probability distribution generation, and CORS middleware for frontend communication.

```python
"""
main.py - FastAPI Backend for Speech Emotion Detection.

Endpoints:
  GET  /health         — Health check, model status
  POST /predict        — Upload audio, get emotion prediction
  GET  /emotions       — List supported emotion classes
  GET  /model-info     — Model metadata

Run with:
  python backend/main.py
  or
  uvicorn backend.main:app --host 127.0.0.1 --port 8000 --reload
"""

import sys
import json
import logging
import tempfile
import os
import io
from pathlib import Path
from typing import Dict, List, Optional
from datetime import datetime

import numpy as np
import joblib
from fastapi import FastAPI, File, UploadFile, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse, FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
import uvicorn

# Add src/ to path so we can import our modules
BACKEND_DIR = Path(__file__).resolve().parent
PROJECT_ROOT = BACKEND_DIR.parent
SRC_DIR = PROJECT_ROOT / "src"
sys.path.insert(0, str(SRC_DIR))

from config import (
    MODEL_PATH, SCALER_PATH, LABEL_ENCODER_PATH, METADATA_PATH,
    SAMPLE_RATE, MIN_AUDIO_DURATION, MAX_AUDIO_SIZE_MB, BACKEND_HOST, BACKEND_PORT
)
from feature_extraction import extract_features_from_bytes, extract_features_from_waveform
from preprocessing import preprocess_audio_bytes, validate_audio

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)

# ── FastAPI App ──────────────────────────────────────────────────
app = FastAPI(
    title="Speech Emotion Detection API",
    description="Detect human emotions from spoken audio using Machine Learning.",
    version="1.0.0",
    docs_url="/docs",
    redoc_url="/redoc",
)

# Allow frontend to call the API from any origin (for local development)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],     # In production, restrict to your frontend domain
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# ── Model storage (loaded once at startup) ──────────────────────
class ModelStore:
    """Holds the loaded model, scaler, and label encoder."""
    model = None
    scaler = None
```

```
    label_encoder = None
    metadata: Dict = {}
    is_loaded: bool = False
    load_error: Optional[str] = None


model_store = ModelStore()


def load_model() -> bool:
    """
    Load the trained model from disk.
    Called once at startup — not on every request.
    """
    global model_store

    required_files = {
        "Model": MODEL_PATH,
        "Scaler": SCALER_PATH,
        "Label Encoder": LABEL_ENCODER_PATH,
    }

    for name, path in required_files.items():
        if not Path(path).exists():
            msg = (
                f"Missing {name}: {path}. "
                f"Please run 'python src/train.py' first to train the model."
            )
            logger.error(msg)
            model_store.load_error = msg
            model_store.is_loaded = False
            return False

    try:
        logger.info("Loading model artifacts...")
        model_store.model         = joblib.load(MODEL_PATH)
        model_store.scaler        = joblib.load(SCALER_PATH)
        model_store.label_encoder = joblib.load(LABEL_ENCODER_PATH)

        if METADATA_PATH.exists():
            with open(METADATA_PATH) as f:
                model_store.metadata = json.load(f)

        model_store.is_loaded = True
        model_store.load_error = None

        classes = list(model_store.label_encoder.classes_)
        logger.info(f"✅ Model loaded: {model_store.metadata.get('model_type', 'Unknown')}")
        logger.info(f"   Emotion classes: {classes}")
        logger.info(f"   Test accuracy: {model_store.metadata.get('test_accuracy', 'N/A')}")
        return True

    except Exception as e:
        msg = f"Failed to load model: {e}"
        logger.error(msg)
        model_store.load_error = msg
        model_store.is_loaded = False
        return False


def determine_audio_format(filename: str, content_type: str) -> str:
    """
    Determine audio format from filename/content-type.
    Browser-recorded audio is often WebM/Opus.
    """
    # Check content type first
    ct_map = {
        "audio/webm": "webm",
        "audio/ogg": "ogg",
        "audio/wav": "wav",
        "audio/x-wav": "wav",
        "audio/mpeg": "mp3",
        "audio/mp4": "mp4",
        "audio/flac": "flac",
        "video/webm": "webm",  # Some browsers send video/webm for audio
    }
    for ct, fmt in ct_map.items():
        if ct in content_type.lower():
            return fmt

    # Fall back to file extension
    ext = Path(filename).suffix.lower().lstrip('.')
    if ext in ['wav', 'mp3', 'ogg', 'flac', 'webm', 'm4a', 'mp4']:
        return ext

    return "webm"  # Default: assume browser recording
```

```python


def predict_emotion(audio_bytes: bytes, audio_format: str) -> Dict:
    """
    Core prediction function.

    Pipeline:
    1. Write bytes to temp file
    2. Preprocess (mono, resample, normalize, pad/trim)
    3. Extract features (MFCC, delta, ZCR, RMS, centroid, rolloff, chroma)
    4. Scale features using saved StandardScaler
    5. Run ML model prediction
    6. Get probability estimates
    7. Return structured result

    Args:
        audio_bytes: Raw audio bytes from upload
        audio_format: Audio format string ('webm', 'wav', etc.)

    Returns:
        Dict with emotion, confidence, probabilities
    """
    if not model_store.is_loaded:
        raise ValueError("Model not loaded. Run training first.")

    # Step 1-3: Preprocess and extract features
    features = extract_features_from_bytes(audio_bytes, audio_format)
    if features is None:
        raise ValueError(
            "Feature extraction failed. The audio may be too short, silent, "
            "or in an unsupported format."
        )

    # Step 4: Scale features (MUST use the scaler fitted during training)
    features_scaled = model_store.scaler.transform(features.reshape(1, -1))

    # Step 5: Predict emotion label
    predicted_label = model_store.model.predict(features_scaled)[0]

    # Step 6: Get probabilities
    # SVM with probability=True uses Platt scaling (calibrated sigmoid)
    # Random Forest uses vote fractions
    # These are the model's confidence estimates, not perfect calibrated probabilities
    try:
        proba = model_store.model.predict_proba(features_scaled)[0]
        classes = model_store.label_encoder.classes_
        probabilities = {cls: float(round(prob, 4)) for cls, prob in zip(classes, proba)}
        confidence = float(max(proba))
    except AttributeError:
        # Fallback if model doesn't support predict_proba
        # Use a one-hot confidence (1.0 for predicted class)
        classes = model_store.label_encoder.classes_
        probabilities = {cls: 0.0 for cls in classes}
        probabilities[predicted_label] = 1.0
        confidence = 1.0
        logger.warning("Model doesn't support probability estimates; using binary confidence.")

    return {
        "emotion": predicted_label,
        "confidence": round(confidence, 4),
        "probabilities": probabilities,
        "timestamp": datetime.now().isoformat(),
    }


# ── Pydantic response models ──────────────────────────────────────────
class HealthResponse(BaseModel):
    status: str
    model_loaded: bool
    model_type: Optional[str] = None
    emotion_classes: Optional[List[str]] = None
    test_accuracy: Optional[float] = None
    error: Optional[str] = None


class PredictionResponse(BaseModel):
    emotion: str
    confidence: float
    probabilities: Dict[str, float]
    timestamp: str


# ── API Endpoints ─────────────────────────────────────────────────────

@app.on_event("startup")
async def startup_event():
```

```
    """Load model when the server starts."""
    logger.info("Starting Speech Emotion Detection API...")
    load_model()


@app.get("/health", response_model=HealthResponse, summary="Health check")
async def health_check():
    """
    Check if the API is running and the model is loaded.

    Returns model status, emotion classes, and accuracy info.
    """
    if model_store.is_loaded:
        return HealthResponse(
            status="ok",
            model_loaded=True,
            model_type=model_store.metadata.get("model_type"),
            emotion_classes=list(model_store.label_encoder.classes_),
            test_accuracy=model_store.metadata.get("test_accuracy"),
        )
    else:
        return HealthResponse(
            status="degraded",
            model_loaded=False,
            error=model_store.load_error or "Model not loaded",
        )


@app.post("/reload-model", summary="Hot-reload model artifacts from disk")
async def reload_model_endpoint():
    """Hot-reload model, scaler, and label encoder without restarting the server."""
    success = load_model()
    if success:
        return {
            "status": "success",
            "message": "Model reloaded successfully",
            "model_type": model_store.metadata.get("model_type"),
            "test_accuracy": model_store.metadata.get("test_accuracy"),
            "feature_dimension": model_store.metadata.get("feature_dimension", 560),
        }
    else:
        return JSONResponse(
            status_code=500,
            content={"status": "error", "message": model_store.load_error or "Failed to reload model"}
        )


@app.get("/emotions", summary="List supported emotions")
async def get_emotions():
    """Return the list of emotion classes the model can detect."""
    if not model_store.is_loaded:
        raise HTTPException(status_code=503, detail="Model not loaded. Run training first.")

    return {
        "emotions": list(model_store.label_encoder.classes_),
        "count": len(model_store.label_encoder.classes_),
    }


@app.get("/model-info", summary="Model metadata")
async def get_model_info():
    """Return detailed metadata about the trained model."""
    if not model_store.is_loaded:
        raise HTTPException(status_code=503, detail="Model not loaded. Run training first.")

    return model_store.metadata


@app.post("/predict", response_model=PredictionResponse, summary="Predict emotion from audio")
async def predict(audio: UploadFile = File(..., description="Audio file (WAV, WebM, OGG, FLAC, MP3)")):
    """
    Predict the emotion in a spoken audio recording.

    Accepts audio in any format (WAV, WebM/Opus from browser, OGG, FLAC, MP3).

    The audio is processed through the same pipeline used during training:
    1. Format conversion (if needed)
    2. Mono conversion + resampling to 22050 Hz
    3. Silence removal + normalization
    4. Fixed-length padding/trimming to 3 seconds
    5. MFCC + additional feature extraction
    6. Feature scaling
    7. ML model prediction

    Returns:
        emotion: Predicted emotion label
```

```
        confidence: Model confidence score [0.0, 1.0]
        probabilities: Score for each emotion class (sums to ~1.0)
    """
    if not model_store.is_loaded:
        raise HTTPException(
            status_code=503,
            detail=f"Model not ready. {model_store.load_error or 'Run python src/train.py first.'}"
        )

    # Validate file size
    audio_bytes = await audio.read()
    file_size_mb = len(audio_bytes) / (1024 * 1024)

    if len(audio_bytes) == 0:
        raise HTTPException(status_code=400, detail="Empty audio file received.")

    if file_size_mb > MAX_AUDIO_SIZE_MB:
        raise HTTPException(
            status_code=413,
            detail=f"File too large: {file_size_mb:.1f} MB. Maximum allowed: {MAX_AUDIO_SIZE_MB} MB."
        )

    # Determine audio format
    filename = audio.filename or "recording.webm"
    content_type = audio.content_type or "audio/webm"
    audio_format = determine_audio_format(filename, content_type)

    logger.info(
        f"Received audio: {filename}, format={audio_format}, "
        f"size={file_size_mb:.2f} MB, content_type={content_type}"
    )

    # Quick duration check using librosa (without full preprocessing)
    try:
        import librosa
        import soundfile as sf
        with tempfile.NamedTemporaryFile(suffix=f".{audio_format}", delete=False) as tmp:
            tmp.write(audio_bytes)
            tmp_path = tmp.name
        try:
            y_check, sr_check = librosa.load(tmp_path, sr=None, mono=True)
            duration = len(y_check) / sr_check
            logger.info(f"Audio duration: {duration:.2f}s, sample rate: {sr_check} Hz")
            if duration < MIN_AUDIO_DURATION:
                raise HTTPException(
                    status_code=400,
                    detail=f"Recording too short: {duration:.1f}s. Please speak for at least {MIN_AUDIO_DURATION}s."
                )
        finally:
            try:
                os.unlink(tmp_path)
            except Exception:
                pass
    except HTTPException:
        raise
    except Exception as e:
        logger.warning(f"Duration check failed: {e} — proceeding with prediction anyway.")

    # Run prediction
    try:
        result = predict_emotion(audio_bytes, audio_format)
        logger.info(
            f"Prediction: {result['emotion']} "
            f"(confidence: {result['confidence']:.2%})"
        )
        return PredictionResponse(**result)

    except ValueError as e:
        raise HTTPException(status_code=422, detail=str(e))
    except Exception as e:
        logger.error(f"Prediction error: {e}", exc_info=True)
        raise HTTPException(
            status_code=500,
            detail="Internal error during emotion prediction. Please try again."
        )


@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    """Catch-all exception handler — never expose stack traces to users."""
    logger.error(f"Unhandled exception: {exc}", exc_info=True)
    return JSONResponse(
        status_code=500,
        content={"detail": "An unexpected error occurred. Please try again."}
    )
```

```python

# ── Serve Frontend ──────────────────────────────────────────────
FRONTEND_DIR = PROJECT_ROOT / "frontend"
if FRONTEND_DIR.exists():
    app.mount("/static", StaticFiles(directory=str(FRONTEND_DIR)), name="static")

    @app.get("/", include_in_schema=False)
    async def serve_index():
        return FileResponse(FRONTEND_DIR / "index.html")

    @app.get("/style.css", include_in_schema=False)
    async def serve_css():
        return FileResponse(FRONTEND_DIR / "style.css", media_type="text/css")

    @app.get("/app.js", include_in_schema=False)
    async def serve_js():
        return FileResponse(FRONTEND_DIR / "app.js", media_type="application/javascript")


# ── Main entry point ────────────────────────────────────────────
if __name__ == "__main__":
    uvicorn.run(
        "main:app",
        host=BACKEND_HOST,
        port=BACKEND_PORT,
        reload=False,
        log_level="info",
    )
```

4. Frontend Web Application

Modern responsive dashboard, Web Audio API recording, real-time waveform visualizer (frontend/)

HTML **frontend/index.html** 297 Lines • 13,078 bytes

Description & Role: Semantic HTML5 web interface featuring dual audio input modalities (live microphone recording with real-time audio canvas visualization and drag-and-drop WAV/MP3 file upload), primary emotion result card with dynamic emotion emojis, confidence percentage bar, emotion probability breakdown meters, audio playback controls, and model architecture info panel.

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>🎙️ Speech Emotion Detection | AI Voice Analysis</title>
  <meta name="description" content="Detect human emotions from your voice using AI and machine learning. Real-time speech emoti
  <meta name="keywords" content="speech emotion recognition, voice analysis, AI, machine learning, emotion detection" />
  <link rel="preconnect" href="https://fonts.googleapis.com" />
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&family=Space+Grotesk:wght@400;500;600
  <link rel="stylesheet" href="style.css" />
</head>
<body>

  <!-- ═══════════════════════════════════════════════════════
       HEADER
  ═══════════════════════════════════════════════════════ -->
  <header class="app-header">
    <div class="header-inner">
      <div class="logo">
        <span class="logo-icon">🎙️</span>
        <div class="logo-text">
          <span class="logo-title">Speech Emotion Detection</span>
          <span class="logo-sub">Powered by Machine Learning</span>
        </div>
      </div>
      <nav class="header-nav">
        <a href="#recorder" class="nav-link active" id="nav-recorder">Record</a>
        <a href="#history" class="nav-link" id="nav-history">History</a>
        <a href="#about" class="nav-link" id="nav-about">About</a>
      </nav>
      <div class="model-badge" id="modelBadge">
        <span class="badge-dot"></span>
        <span id="modelStatus">Connecting...</span>
      </div>
    </div>
  </header>

  <!-- ═══════════════════════════════════════════════════════
       HERO / RECORDER SECTION
  ═══════════════════════════════════════════════════════ -->
  <main>
    <section class="hero" id="recorder">
      <div class="hero-text">
        <h1 class="hero-title">Speak. We'll detect the emotion.</h1>
        <p class="hero-subtitle">Speak naturally and let AI analyze the emotion in your voice.</p>
      </div>

      <!-- ── Recorder Card ── -->
      <div class="recorder-card" id="recorderCard">

        <!-- State: Idle -->
        <div class="record-state" id="stateIdle">
          <div class="mic-container" id="micContainer">
            <button class="mic-btn" id="startBtn" title="Start Recording" aria-label="Start Recording">
              <svg class="mic-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8">
                <path d="M12 1a4 4 0 0 1 4 4v6a4 4 0 0 1-8 0V5a4 4 0 0 1 4-4z"/>
                <path d="M19 10v2a7 7 0 0 1-14 0v-2"/>
                <line x1="12" y1="19" x2="12" y2="23"/>
                <line x1="8" y1="23" x2="16" y2="23"/>
              </svg>
            </button>
            <div class="mic-rings">
              <div class="ring ring-1"></div>
              <div class="ring ring-2"></div>
              <div class="ring ring-3"></div>
            </div>
          </div>
          <p class="record-hint">Click the microphone and speak</p>
          <p class="record-sub">Recommended: speak 2–5 seconds</p>
        </div>

        <!-- State: Recording -->
        <div class="record-state hidden" id="stateRecording">
          <div class="recording-indicator">
```

```
            <div class="rec-dot"></div>
            <span class="rec-label">Recording...</span>
          </div>
          <div class="timer" id="recordTimer">00:00</div>
          <div class="waveform" id="waveform" aria-hidden="true">
            <div class="wave-bar"></div><div class="wave-bar"></div>
            <div class="wave-bar"></div><div class="wave-bar"></div>
            <div class="wave-bar"></div><div class="wave-bar"></div>
            <div class="wave-bar"></div><div class="wave-bar"></div>
            <div class="wave-bar"></div><div class="wave-bar"></div>
            <div class="wave-bar"></div><div class="wave-bar"></div>
            <div class="wave-bar"></div>
          </div>
          <button class="stop-btn" id="stopBtn" aria-label="Stop Recording">
            <svg viewBox="0 0 24 24" fill="currentColor" width="20" height="20">
              <rect x="6" y="6" width="12" height="12" rx="2"/>
            </svg>
            Stop Recording
          </button>
        </div>

        <!-- State: Processing -->
        <div class="record-state hidden" id="stateProcessing">
          <div class="processing-spinner">
            <div class="spinner-ring"></div>
            <span class="spinner-icon">🧠</span>
          </div>
          <p class="processing-text">Analyzing your voice...</p>
          <p class="processing-sub">Extracting MFCC features &amp;amp; running the ML model</p>
        </div>

        <!-- State: Error -->
        <div class="record-state hidden" id="stateError">
          <div class="error-icon">⚠️</div>
          <p class="error-title" id="errorTitle">Something went wrong</p>
          <p class="error-message" id="errorMessage">Please try again.</p>
          <button class="retry-btn" id="retryBtn">Try Again</button>
        </div>

        <!-- State: Result -->
        <div class="record-state hidden" id="stateResult">
          <div class="result-emotion-display">
            <div class="emotion-icon-big" id="resultEmoji">😊</div>
            <div class="emotion-label" id="resultEmotion">HAPPY</div>
            <div class="confidence-row">
              <span class="conf-label">Confidence</span>
              <div class="conf-bar-wrap">
                <div class="conf-bar" id="confBar"></div>
              </div>
              <span class="conf-value" id="confValue">87%</span>
            </div>
          </div>

          <!-- Emotion Probability Bars -->
          <div class="prob-section">
            <h3 class="prob-title">Emotion Analysis</h3>
            <div class="prob-bars" id="probBars">
              <!-- Filled dynamically by JavaScript -->
            </div>
          </div>

          <button class="record-again-btn" id="recordAgainBtn">
            🎙 Record Again
          </button>
        </div>

      </div><!-- /recorder-card -->

      <!-- Privacy Note -->
      <p class="privacy-note" id="privacyNote">
        🔒 The microphone is used <strong>only</strong> when you start recording. Audio is sent to the application
        for emotion prediction and is <strong>not permanently stored</strong>.
      </p>

    </section>

    <!-- ═══════════════════════════════════════════════════════
         HISTORY SECTION
    ═══════════════════════════════════════════════════════ -->
    <section class="section" id="history">
      <div class="section-header">
        <h2 class="section-title">Recent Predictions</h2>
        <button class="clear-btn" id="clearHistoryBtn">Clear History</button>
      </div>
      <div class="history-list" id="historyList">
```

```
        <div class="history-empty" id="historyEmpty">
          <span class="history-empty-icon">🗂️</span>
          <p>No predictions yet. Record your voice to get started!</p>
        </div>
      </div>
    </section>

    <!-- ═══════════════════════════════════════════════════════
         ABOUT SECTION
    ═══════════════════════════════════════════════════════ -->
    <section class="section" id="about">
      <h2 class="section-title">About This System</h2>

      <div class="about-grid">
        <div class="about-card">
          <h3>🧠 What is Speech Emotion Recognition?</h3>
          <p>
            Speech Emotion Recognition (SER) is the process of automatically identifying
            a human's emotional state from their speech signal. The voice carries rich
            information beyond words — pitch, rhythm, energy, and tone all reflect
            emotional states like happiness, anger, sadness, and more.
          </p>
        </div>

        <div class="about-card">
          <h3>📊 What is MFCC?</h3>
          <p>
            <strong>Mel Frequency Cepstral Coefficients (MFCCs)</strong> are the primary
            feature for speech processing. They represent how the human ear perceives
            different frequencies (logarithmic mel scale). MFCCs capture the vocal tract
            shape that differs between emotional states — making them perfect for SER.
          </p>
        </div>
      </div>

      <!-- Pipeline -->
      <div class="pipeline-card">
        <h3 class="pipeline-title">How It Works</h3>
        <div class="pipeline">
          <div class="pipeline-step">
            <div class="pipe-icon">🎤</div>
            <div class="pipe-label">Voice Input</div>
            <div class="pipe-desc">Browser microphone captures your speech</div>
          </div>
          <div class="pipe-arrow">→</div>
          <div class="pipeline-step">
            <div class="pipe-icon">🔊</div>
            <div class="pipe-label">Audio Processing</div>
            <div class="pipe-desc">Mono conversion, resampling to 22050 Hz, normalization</div>
          </div>
          <div class="pipe-arrow">→</div>
          <div class="pipeline-step">
            <div class="pipe-icon">📊</div>
            <div class="pipe-label">Feature Extraction</div>
            <div class="pipe-desc">MFCC, Delta-MFCC, ZCR, RMS, Spectral features (192 values)</div>
          </div>
          <div class="pipe-arrow">→</div>
          <div class="pipeline-step">
            <div class="pipe-icon">🤖</div>
            <div class="pipe-label">ML Model</div>
            <div class="pipe-desc">Trained SVM/Random Forest on RAVDESS dataset</div>
          </div>
          <div class="pipe-arrow">→</div>
          <div class="pipeline-step">
            <div class="pipe-icon">🧠</div>
            <div class="pipe-label">Classification</div>
            <div class="pipe-desc">8-class emotion prediction with confidence scores</div>
          </div>
          <div class="pipe-arrow">→</div>
          <div class="pipeline-step">
            <div class="pipe-icon">📱</div>
            <div class="pipe-label">Result</div>
            <div class="pipe-desc">Emotion label + probability visualization</div>
          </div>
        </div>
      </div>

      <!-- Feature Table -->
      <div class="feature-table-card">
        <h3>Audio Features Used</h3>
        <div class="table-scroll">
          <table class="feature-table">
            <thead>
              <tr>
                <th>Feature</th>
                <th>Dimension</th>
```

```
                <th>What it captures</th>
              </tr>
            </thead>
            <tbody>
              <tr><td>MFCC (40 coefficients)</td><td>80</td><td>Vocal tract shape — primary SER feature</td></tr>
              <tr><td>MFCC Delta</td><td>80</td><td>Rate of change — speech dynamics</td></tr>
              <tr><td>Zero Crossing Rate</td><td>2</td><td>Noisiness / voiced vs. unvoiced segments</td></tr>
              <tr><td>RMS Energy</td><td>2</td><td>Loudness — linked to emotional intensity</td></tr>
              <tr><td>Spectral Centroid</td><td>2</td><td>"Brightness" of sound</td></tr>
              <tr><td>Spectral Rolloff</td><td>2</td><td>Tonal characteristic of speech</td></tr>
              <tr><td>Chroma</td><td>24</td><td>Pitch class energy — tonal content</td></tr>
              <tr class="total-row"><td><strong>Total</strong></td><td><strong>192</strong></td><td>Fixed-size feature vector</
            </tbody>
          </table>
        </div>
      </div>

      <!-- Emotions -->
      <div class="emotions-card">
        <h3>Supported Emotions (RAVDESS Dataset)</h3>
        <div class="emotions-grid" id="emotionsGrid">
          <!-- Filled by JS after loading model info -->
        </div>
      </div>

      <!-- Limitations -->
      <div class="about-card limitations-card">
        <h3>⚠️ Limitations &amp;amp; Disclaimer</h3>
        <ul>
          <li>Emotion recognition from voice is <strong>probabilistic</strong>, not certain.</li>
          <li>Background noise can significantly affect prediction accuracy.</li>
          <li>The model was trained on RAVDESS (English, professional actors) — it may not generalize to all accents, languages
          <li>Short recordings (&lt; 2 seconds) may yield less reliable predictions.</li>
          <li>This system detects <strong>expressed speech emotion</strong>, not true psychological state.</li>
          <li>Microphone quality affects audio quality and therefore prediction accuracy.</li>
        </ul>
      </div>
    </section>
  </main>

  <!-- Footer -->
  <footer class="app-footer">
    <p>🎙️ Speech Emotion Detection | RAVDESS Dataset | Built with FastAPI + scikit-learn</p>
    <p class="footer-sub">B.E. Computer Science Project | Machine Learning System</p>
  </footer>

  <script src="app.js"></script>
</body>
</html>
```

CSS **frontend/style.css** 573 Lines • 19,274 bytes

Description & Role: State-of-the-art CSS3 styling featuring dark-mode design system, glassmorphism UI cards with backdrop blur, custom CSS variables, responsive grid layout, emotion-specific color schemes (Angry, Happy, Sad, Calm, Fearful, Disgust, Surprised, Neutral), smooth pulse/ripple animations for microphone recording, and accessible typography.

```css
/* ═══════════════════════════════════════════════════
   Speech Emotion Detection — style.css
   Dark glassmorphism design with smooth animations
═══════════════════════════════════════════════════ */

/* ── CSS Variables ── */
:root {
  --bg-primary:    #0d0d1a;
  --bg-secondary:  #111128;
  --bg-card:       rgba(255, 255, 255, 0.04);
  --bg-card-hover: rgba(255, 255, 255, 0.07);
  --border:        rgba(255, 255, 255, 0.08);
  --border-active: rgba(99, 102, 241, 0.5);

  --accent:        #6366f1;
  --accent-light:  #818cf8;
  --accent-glow:   rgba(99, 102, 241, 0.3);
  --danger:        #ef4444;
  --success:       #22c55e;
  --warning:       #f59e0b;

  --text-primary:  #f1f5f9;
  --text-secondary:#94a3b8;
  --text-muted:    #64748b;

  /* Emotion colors */
  --neutral:       #94a3b8;
  --calm:          #60a5fa;
  --happy:         #fbbf24;
  --sad:           #6366f1;
  --angry:         #ef4444;
  --fearful:       #a855f7;
  --disgust:       #22c55e;
  --surprised:     #f97316;

  --radius-sm: 8px;
  --radius:    16px;
  --radius-lg: 24px;
  --shadow:    0 4px 24px rgba(0, 0, 0, 0.4);
  --shadow-lg: 0 8px 48px rgba(0, 0, 0, 0.6);

  --font-body: 'Inter', sans-serif;
  --font-display: 'Space Grotesk', sans-serif;

  --transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
}

/* ── Reset ── */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
html { scroll-behavior: smooth; }
body {
  background: var(--bg-primary);
  color: var(--text-primary);
  font-family: var(--font-body);
  font-size: 15px;
  line-height: 1.6;
  min-height: 100vh;
  overflow-x: hidden;
}

/* Background gradient blobs */
body::before, body::after {
  content: '';
  position: fixed;
  border-radius: 50%;
  filter: blur(80px);
  pointer-events: none;
  z-index: 0;
}
body::before {
  width: 600px; height: 600px;
  background: radial-gradient(circle, rgba(99,102,241,0.12) 0%, transparent 70%);
  top: -200px; left: -200px;
}
body::after {
  width: 500px; height: 500px;
```

```
  background: radial-gradient(circle, rgba(168,85,247,0.1) 0%, transparent 70%);
  bottom: -150px; right: -150px;
}

/* ── Header ── */
.app-header {
  position: sticky; top: 0; z-index: 100;
  background: rgba(13, 13, 26, 0.85);
  backdrop-filter: blur(20px);
  -webkit-backdrop-filter: blur(20px);
  border-bottom: 1px solid var(--border);
  padding: 0 24px;
}
.header-inner {
  max-width: 1100px;
  margin: 0 auto;
  height: 64px;
  display: flex;
  align-items: center;
  gap: 24px;
}
.logo {
  display: flex; align-items: center; gap: 12px;
  text-decoration: none; color: inherit;
}
.logo-icon { font-size: 1.6rem; }
.logo-title { font-family: var(--font-display); font-size: 1rem; font-weight: 700; color: var(--text-primary); display: block; }
.logo-sub   { font-size: 0.7rem; color: var(--text-muted); display: block; }
.header-nav { display: flex; gap: 4px; margin-left: auto; }
.nav-link {
  padding: 6px 16px; border-radius: 20px;
  text-decoration: none; color: var(--text-secondary);
  font-size: 0.85rem; font-weight: 500;
  transition: var(--transition);
}
.nav-link:hover, .nav-link.active {
  background: rgba(99,102,241,0.15);
  color: var(--accent-light);
}
.model-badge {
  display: flex; align-items: center; gap: 8px;
  background: rgba(255,255,255,0.05);
  border: 1px solid var(--border);
  border-radius: 20px;
  padding: 5px 14px;
  font-size: 0.78rem; color: var(--text-secondary);
  white-space: nowrap;
}
.badge-dot {
  width: 7px; height: 7px; border-radius: 50%;
  background: var(--text-muted);
  transition: var(--transition);
}
.badge-dot.online  { background: var(--success); box-shadow: 0 0 6px var(--success); }
.badge-dot.offline { background: var(--danger); }
.badge-dot.loading {
  background: var(--warning);
  animation: pulse 1s infinite;
}

/* ── Hero ── */
.hero {
  max-width: 760px;
  margin: 0 auto;
  padding: 60px 24px 40px;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 32px;
  position: relative; z-index: 1;
}
.hero-text { text-align: center; }
.hero-title {
  font-family: var(--font-display);
  font-size: clamp(1.8rem, 4vw, 2.8rem);
  font-weight: 700;
  line-height: 1.15;
  background: linear-gradient(135deg, #f1f5f9 0%, #818cf8 60%, #a855f7 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
  margin-bottom: 12px;
}
.hero-subtitle {
  font-size: 1.05rem; color: var(--text-secondary);
  max-width: 480px; margin: 0 auto;
```

```
}

/* ── Recorder Card ── */
.recorder-card {
  width: 100%;
  max-width: 600px;
  background: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding: 48px 40px;
  backdrop-filter: blur(12px);
  box-shadow: var(--shadow-lg);
  min-height: 320px;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  position: relative;
  overflow: hidden;
  transition: var(--transition);
}
.recorder-card::before {
  content: '';
  position: absolute;
  inset: 0;
  border-radius: var(--radius-lg);
  background: linear-gradient(135deg, rgba(99,102,241,0.05) 0%, transparent 60%);
  pointer-events: none;
}
.record-state {
  display: flex; flex-direction: column; align-items: center;
  gap: 20px; width: 100%;
  animation: fadeIn 0.3s ease;
}
.record-state.hidden { display: none !important; }

/* ── Microphone Button ── */
.mic-container {
  position: relative;
  display: flex; align-items: center; justify-content: center;
}
.mic-btn {
  width: 100px; height: 100px; border-radius: 50%;
  background: linear-gradient(135deg, var(--accent) 0%, #a855f7 100%);
  border: none; cursor: pointer;
  display: flex; align-items: center; justify-content: center;
  box-shadow: 0 0 30px var(--accent-glow), 0 4px 20px rgba(0,0,0,0.4);
  transition: var(--transition);
  position: relative; z-index: 2;
}
.mic-btn:hover {
  transform: scale(1.08);
  box-shadow: 0 0 50px rgba(99,102,241,0.5), 0 4px 24px rgba(0,0,0,0.5);
}
.mic-btn:active { transform: scale(0.96); }
.mic-icon { width: 40px; height: 40px; stroke: white; }

/* Pulsing rings */
.mic-rings { position: absolute; inset: 0; }
.ring {
  position: absolute;
  border-radius: 50%;
  border: 2px solid var(--accent);
  opacity: 0;
  animation: ring-pulse 3s infinite;
}
.ring-1 { inset: -10px; animation-delay: 0s; }
.ring-2 { inset: -24px; animation-delay: 0.8s; }
.ring-3 { inset: -38px; animation-delay: 1.6s; }

@keyframes ring-pulse {
  0%   { opacity: 0.6; transform: scale(0.95); }
  100% { opacity: 0; transform: scale(1.15); }
}

.record-hint { font-size: 1rem; color: var(--text-primary); font-weight: 500; }
.record-sub  { font-size: 0.82rem; color: var(--text-muted); }

/* ── Recording State ── */
.recording-indicator {
  display: flex; align-items: center; gap: 10px;
  font-size: 0.95rem; font-weight: 600; color: var(--danger);
}
.rec-dot {
  width: 10px; height: 10px; border-radius: 50%;
  background: var(--danger);
```

```
  animation: blink 1s infinite;
}
@keyframes blink { 0%,100% { opacity: 1; } 50% { opacity: 0.2; } }
.timer {
  font-family: var(--font-display);
  font-size: 2.5rem; font-weight: 700;
  color: var(--text-primary);
  letter-spacing: 2px;
}

/* Waveform */
.waveform {
  display: flex; align-items: center; gap: 3px;
  height: 48px;
}
.wave-bar {
  width: 3px; border-radius: 2px;
  background: linear-gradient(to top, var(--accent), #a855f7);
  animation: wave-anim 0.6s infinite ease-in-out alternate;
  min-height: 4px;
}
.wave-bar:nth-child(1)  { animation-delay: 0.0s; }
.wave-bar:nth-child(2)  { animation-delay: 0.05s; }
.wave-bar:nth-child(3)  { animation-delay: 0.10s; }
.wave-bar:nth-child(4)  { animation-delay: 0.15s; }
.wave-bar:nth-child(5)  { animation-delay: 0.20s; }
.wave-bar:nth-child(6)  { animation-delay: 0.25s; }
.wave-bar:nth-child(7)  { animation-delay: 0.30s; }
.wave-bar:nth-child(8)  { animation-delay: 0.25s; }
.wave-bar:nth-child(9)  { animation-delay: 0.20s; }
.wave-bar:nth-child(10) { animation-delay: 0.15s; }
.wave-bar:nth-child(11) { animation-delay: 0.10s; }
.wave-bar:nth-child(12) { animation-delay: 0.05s; }
.wave-bar:nth-child(13) { animation-delay: 0.0s; }
.wave-bar:nth-child(14) { animation-delay: 0.05s; }
.wave-bar:nth-child(15) { animation-delay: 0.10s; }

@keyframes wave-anim {
  from { height: 4px; }
  to   { height: 40px; }
}

.stop-btn {
  display: flex; align-items: center; gap: 8px;
  padding: 12px 28px; border-radius: 12px;
  background: var(--danger);
  border: none; cursor: pointer;
  color: white; font-size: 0.95rem; font-weight: 600;
  transition: var(--transition);
  box-shadow: 0 4px 16px rgba(239, 68, 68, 0.3);
}
.stop-btn:hover { background: #dc2626; transform: translateY(-1px); }

/* ── Processing State ── */
.processing-spinner {
  position: relative;
  width: 90px; height: 90px;
  display: flex; align-items: center; justify-content: center;
}
.spinner-ring {
  position: absolute; inset: 0; border-radius: 50%;
  border: 3px solid transparent;
  border-top-color: var(--accent);
  border-right-color: #a855f7;
  animation: spin 1.2s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }
.spinner-icon { font-size: 2rem; }
.processing-text { font-size: 1.1rem; font-weight: 600; color: var(--text-primary); }
.processing-sub  { font-size: 0.82rem; color: var(--text-muted); text-align: center; }

/* ── Error State ── */
.error-icon { font-size: 3rem; }
.error-title { font-size: 1.1rem; font-weight: 600; color: var(--danger); }
.error-message { font-size: 0.88rem; color: var(--text-secondary); text-align: center; max-width: 320px; }
.retry-btn {
  padding: 10px 28px; border-radius: 10px;
  background: rgba(99,102,241,0.15);
  border: 1px solid var(--accent);
  color: var(--accent-light); cursor: pointer;
  font-size: 0.9rem; font-weight: 600;
  transition: var(--transition);
}
.retry-btn:hover { background: rgba(99,102,241,0.25); }

/* ── Result State ── */
```

```
.result-emotion-display {
  text-align: center;
  display: flex; flex-direction: column; align-items: center; gap: 12px;
  width: 100%;
}
.emotion-icon-big { font-size: 4rem; animation: pop-in 0.4s cubic-bezier(0.175,0.885,0.32,1.275); }
@keyframes pop-in { from { transform: scale(0.5); opacity: 0; } to { transform: scale(1); opacity: 1; } }
.emotion-label {
  font-family: var(--font-display);
  font-size: 1.8rem; font-weight: 800;
  letter-spacing: 3px;
  background: linear-gradient(135deg, var(--accent-light), #a855f7);
  -webkit-background-clip: text; -webkit-text-fill-color: transparent;
  background-clip: text;
}
.confidence-row {
  display: flex; align-items: center; gap: 10px; width: 100%; max-width: 300px;
}
.conf-label { font-size: 0.78rem; color: var(--text-muted); white-space: nowrap; }
.conf-bar-wrap {
  flex: 1; height: 6px; background: rgba(255,255,255,0.08);
  border-radius: 3px; overflow: hidden;
}
.conf-bar {
  height: 100%; border-radius: 3px;
  background: linear-gradient(90deg, var(--accent), #a855f7);
  transition: width 0.8s cubic-bezier(0.4,0,0.2,1);
  width: 0%;
}
.conf-value { font-size: 0.88rem; font-weight: 700; color: var(--accent-light); min-width: 36px; text-align: right; }

/* Probability bars */
.prob-section { width: 100%; padding-top: 8px; border-top: 1px solid var(--border); }
.prob-title { font-size: 0.82rem; font-weight: 600; color: var(--text-muted); text-transform: uppercase; letter-spacing: 1px; m
.prob-bars { display: flex; flex-direction: column; gap: 8px; }
.prob-row {
  display: grid; grid-template-columns: 90px 1fr 46px;
  align-items: center; gap: 8px;
}
.prob-emotion-name { font-size: 0.82rem; color: var(--text-secondary); font-weight: 500; }
.prob-bar-wrap { height: 6px; background: rgba(255,255,255,0.06); border-radius: 3px; overflow: hidden; }
.prob-bar-fill {
  height: 100%; border-radius: 3px;
  transition: width 0.8s cubic-bezier(0.4,0,0.2,1) var(--delay, 0s);
  width: 0%;
}
.prob-pct { font-size: 0.78rem; color: var(--text-muted); text-align: right; }
.prob-pct.top { color: var(--accent-light); font-weight: 700; }

.record-again-btn {
  padding: 11px 28px; border-radius: 12px;
  background: linear-gradient(135deg, var(--accent), #7c3aed);
  border: none; cursor: pointer;
  color: white; font-size: 0.9rem; font-weight: 600;
  transition: var(--transition);
  box-shadow: 0 4px 16px var(--accent-glow);
  margin-top: 8px;
}
.record-again-btn:hover { transform: translateY(-2px); box-shadow: 0 8px 24px var(--accent-glow); }

/* ── Privacy Note ── */
.privacy-note {
  font-size: 0.78rem; color: var(--text-muted);
  text-align: center; max-width: 480px;
  background: rgba(255,255,255,0.03);
  border: 1px solid var(--border);
  border-radius: var(--radius-sm);
  padding: 10px 16px;
}

/* ── Sections ── */
.section {
  max-width: 1000px; margin: 0 auto;
  padding: 60px 24px;
  position: relative; z-index: 1;
}
.section-header {
  display: flex; align-items: center; justify-content: space-between;
  margin-bottom: 28px; flex-wrap: wrap; gap: 12px;
}
.section-title {
  font-family: var(--font-display);
  font-size: 1.5rem; font-weight: 700;
  color: var(--text-primary);
}
.clear-btn {
```

```
  padding: 6px 18px; border-radius: 8px;
  background: rgba(239,68,68,0.1);
  border: 1px solid rgba(239,68,68,0.3);
  color: #f87171; cursor: pointer;
  font-size: 0.82rem; font-weight: 500;
  transition: var(--transition);
}
.clear-btn:hover { background: rgba(239,68,68,0.2); }

/* ── History ── */
.history-list { display: flex; flex-direction: column; gap: 10px; }
.history-empty {
  text-align: center; padding: 48px 24px;
  color: var(--text-muted);
  border: 1px dashed var(--border);
  border-radius: var(--radius);
}
.history-empty-icon { font-size: 2.5rem; display: block; margin-bottom: 12px; }
.history-item {
  display: flex; align-items: center; gap: 16px;
  background: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 14px 20px;
  transition: var(--transition);
  animation: slide-in 0.3s ease;
}
.history-item:hover { background: var(--bg-card-hover); border-color: var(--border-active); }
@keyframes slide-in { from { opacity: 0; transform: translateY(-8px); } to { opacity: 1; transform: translateY(0); } }
.history-emoji { font-size: 1.5rem; flex-shrink: 0; }
.history-info { flex: 1; }
.history-emotion { font-weight: 700; font-size: 0.95rem; }
.history-time { font-size: 0.75rem; color: var(--text-muted); margin-top: 2px; }
.history-conf {
  font-size: 1rem; font-weight: 700;
  background: linear-gradient(135deg, var(--accent-light), #a855f7);
  -webkit-background-clip: text; -webkit-text-fill-color: transparent;
  background-clip: text;
}

/* ── About ── */
.about-grid {
  display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 20px; margin-bottom: 24px;
}
.about-card {
  background: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 28px;
}
.about-card h3 { font-family: var(--font-display); font-size: 1rem; font-weight: 700; margin-bottom: 12px; color: var(--text-pr
.about-card p  { font-size: 0.88rem; color: var(--text-secondary); line-height: 1.7; }
.about-card ul { list-style: none; padding: 0; }
.about-card li { font-size: 0.88rem; color: var(--text-secondary); padding: 6px 0; padding-left: 20px; position: relative; }
.about-card li::before { content: '›'; position: absolute; left: 0; color: var(--warning); font-weight: 700; }

.limitations-card { margin-top: 24px; border-color: rgba(245,158,11,0.3); }
.limitations-card h3 { color: var(--warning); }

/* ── Pipeline ── */
.pipeline-card {
  background: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 32px; margin-bottom: 24px;
  overflow-x: auto;
}
.pipeline-title { font-family: var(--font-display); font-size: 1rem; font-weight: 700; margin-bottom: 24px; }
.pipeline {
  display: flex; align-items: flex-start; gap: 0;
  min-width: 600px;
}
.pipeline-step { flex: 1; text-align: center; padding: 0 4px; }
.pipe-icon { font-size: 1.8rem; margin-bottom: 8px; }
.pipe-label { font-size: 0.8rem; font-weight: 700; color: var(--text-primary); margin-bottom: 6px; }
.pipe-desc  { font-size: 0.72rem; color: var(--text-muted); line-height: 1.4; }
.pipe-arrow { color: var(--accent); font-size: 1.2rem; padding: 0 2px; margin-top: 18px; flex-shrink: 0; }

/* ── Feature Table ── */
.feature-table-card {
  background: var(--bg-card); border: 1px solid var(--border);
  border-radius: var(--radius); padding: 28px; margin-bottom: 24px;
}
.feature-table-card h3 { font-family: var(--font-display); font-size: 1rem; font-weight: 700; margin-bottom: 16px; }
.table-scroll { overflow-x: auto; }
```

```
.feature-table {
  width: 100%; border-collapse: collapse;
  font-size: 0.85rem;
}
.feature-table th {
  text-align: left; padding: 10px 14px;
  background: rgba(255,255,255,0.04);
  color: var(--text-muted); font-weight: 600;
  font-size: 0.78rem; text-transform: uppercase; letter-spacing: 0.5px;
}
.feature-table td { padding: 10px 14px; border-top: 1px solid var(--border); color: var(--text-secondary); }
.feature-table td:first-child { color: var(--text-primary); font-weight: 500; }
.feature-table td:nth-child(2) { color: var(--accent-light); font-weight: 700; font-family: monospace; }
.total-row td { border-top: 2px solid var(--accent) !important; background: rgba(99,102,241,0.05); }

/* ── Emotions Grid ── */
.emotions-card {
  background: var(--bg-card); border: 1px solid var(--border);
  border-radius: var(--radius); padding: 28px; margin-bottom: 24px;
}
.emotions-card h3 { font-family: var(--font-display); font-size: 1rem; font-weight: 700; margin-bottom: 20px; }
.emotions-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(120px, 1fr)); gap: 12px; }
.emotion-chip {
  display: flex; flex-direction: column; align-items: center;
  gap: 8px; padding: 16px 8px;
  background: rgba(255,255,255,0.04); border: 1px solid var(--border);
  border-radius: var(--radius-sm);
  font-size: 0.8rem; font-weight: 600;
  transition: var(--transition);
}
.emotion-chip:hover {
  background: rgba(255,255,255,0.08);
  transform: translateY(-2px);
}
.emotion-chip-icon { font-size: 1.6rem; }

/* ── Footer ── */
.app-footer {
  text-align: center;
  padding: 32px 24px;
  border-top: 1px solid var(--border);
  color: var(--text-muted); font-size: 0.8rem;
  position: relative; z-index: 1;
}
.footer-sub { margin-top: 4px; font-size: 0.72rem; }

/* ── Animations ── */
@keyframes fadeIn { from { opacity: 0; transform: translateY(8px); } to { opacity: 1; transform: translateY(0); } }
@keyframes pulse  { 0%,100% { opacity: 1; } 50% { opacity: 0.4; } }

/* ── Responsive ── */
@media (max-width: 768px) {
  .header-inner { padding: 0 4px; gap: 12px; }
  .logo-title { font-size: 0.88rem; }
  .hero { padding: 40px 16px 32px; }
  .recorder-card { padding: 32px 20px; }
  .model-badge { display: none; }
  .pipeline { flex-direction: column; min-width: unset; align-items: center; }
  .pipe-arrow { transform: rotate(90deg); margin: 0; padding: 4px 0; }
  .pipeline-step { max-width: 220px; }
}
@media (max-width: 480px) {
  .hero-title { font-size: 1.5rem; }
  .section { padding: 40px 16px; }
  .recorder-card { padding: 28px 16px; }
  .mic-btn { width: 84px; height: 84px; }
}
```

JS **frontend/app.js** 526 Lines • 17,689 bytes

Description & Role: Client-side JavaScript application powering Web Audio API microphone stream capture, MediaRecorder audio blob encoding, HTML5 Canvas 60fps real-time waveform visualizer, drag-and-drop file upload handling, asynchronous REST API communication with FastAPI backend, dynamic confidence chart updates, audio playback state management, and error toast notifications.

```
/**
 * app.js — Frontend JavaScript for Speech Emotion Detection
 *
 * Handles:
 * - Microphone recording with MediaRecorder API
 * - Audio upload to FastAPI backend
 * - Displaying prediction results
 * - Prediction history (localStorage)
 * - Backend health check
 * - All error states
 */

'use strict';

// ── Configuration ──────────────────────────────────────────────
const API_BASE = 'http://127.0.0.1:8000';
const HISTORY_KEY = 'sed_prediction_history';
const MAX_HISTORY = 20;
const MIN_RECORDING_MS = 500;  // Minimum recording before stop is allowed

// ── Emotion Metadata (icons + colors for display only)
// The actual emotion label comes from the ML model — this is only for UI visuals
const EMOTION_META = {
  'Neutral':   { emoji: '😐', color: '#94a3b8', bg: 'rgba(148,163,184,0.15)' },
  'Calm':      { emoji: '😌', color: '#60a5fa', bg: 'rgba(96,165,250,0.15)' },
  'Happy':     { emoji: '😊', color: '#fbbf24', bg: 'rgba(251,191,36,0.15)' },
  'Sad':       { emoji: '😢', color: '#6366f1', bg: 'rgba(99,102,241,0.15)' },
  'Angry':     { emoji: '😠', color: '#ef4444', bg: 'rgba(239,68,68,0.15)' },
  'Fearful':   { emoji: '😨', color: '#a855f7', bg: 'rgba(168,85,247,0.15)' },
  'Disgust':   { emoji: '🤢', color: '#22c55e', bg: 'rgba(34,197,94,0.15)' },
  'Surprised': { emoji: '😲', color: '#f97316', bg: 'rgba(249,115,22,0.15)' },
};

// Default for unknown emotions (model may return unlisted label)
function getEmotionMeta(emotion) {
  return EMOTION_META[emotion] || { emoji: '🎭', color: '#818cf8', bg: 'rgba(129,140,248,0.15)' };
}

// ── State ──────────────────────────────────────────────────────
let mediaRecorder = null;
let audioChunks   = [];
let recordingTimer = null;
let recordingStartTime = null;
let currentStream = null;

// ── DOM Elements ───────────────────────────────────────────────
const stateIdle       = document.getElementById('stateIdle');
const stateRecording  = document.getElementById('stateRecording');
const stateProcessing = document.getElementById('stateProcessing');
const stateError      = document.getElementById('stateError');
const stateResult     = document.getElementById('stateResult');

const startBtn        = document.getElementById('startBtn');
const stopBtn         = document.getElementById('stopBtn');
const retryBtn        = document.getElementById('retryBtn');
const recordAgainBtn  = document.getElementById('recordAgainBtn');
const clearHistoryBtn = document.getElementById('clearHistoryBtn');

const recordTimer     = document.getElementById('recordTimer');
const resultEmoji     = document.getElementById('resultEmoji');
const resultEmotion   = document.getElementById('resultEmotion');
const confBar         = document.getElementById('confBar');
const confValue       = document.getElementById('confValue');
const probBars        = document.getElementById('probBars');
const errorTitle      = document.getElementById('errorTitle');
const errorMessage    = document.getElementById('errorMessage');
const historyList     = document.getElementById('historyList');
const historyEmpty    = document.getElementById('historyEmpty');
const modelBadge      = document.getElementById('modelBadge');
const modelStatus     = document.getElementById('modelStatus');
const badgeDot        = modelBadge.querySelector('.badge-dot');
const emotionsGrid    = document.getElementById('emotionsGrid');


// ── State Machine ──────────────────────────────────────────────
function showState(name) {
```

```
  [stateIdle, stateRecording, stateProcessing, stateError, stateResult].forEach(el => {
    el.classList.add('hidden');
  });
  const map = {
    idle: stateIdle, recording: stateRecording,
    processing: stateProcessing, error: stateError, result: stateResult
  };
  if (map[name]) map[name].classList.remove('hidden');
}


// ── Health Check ──────────────────────────────────────────────
async function checkHealth() {
  badgeDot.className = 'badge-dot loading';
  modelStatus.textContent = 'Connecting...';

  try {
    const resp = await fetch(`${API_BASE}/health`, { signal: AbortSignal.timeout(5000) });
    const data = await resp.json();

    if (data.model_loaded) {
      badgeDot.className = 'badge-dot online';
      const acc = data.test_accuracy ? ` · ${(data.test_accuracy * 100).toFixed(1)}% acc` : '';
      modelStatus.textContent = `${data.model_type || 'Model'}${acc}`;

      // Load emotion classes
      if (data.emotion_classes) {
        renderEmotionsGrid(data.emotion_classes);
      }
    } else {
      badgeDot.className = 'badge-dot offline';
      modelStatus.textContent = 'Model not loaded';
      console.warn('Model not loaded:', data.error);
    }
  } catch (err) {
    badgeDot.className = 'badge-dot offline';
    modelStatus.textContent = 'Backend offline';
    console.warn('Backend not reachable:', err.message);
  }
}


// ── Emotions Grid (About section) ─────────────────────────────
function renderEmotionsGrid(emotions) {
  if (!emotionsGrid) return;
  emotionsGrid.innerHTML = '';
  emotions.forEach(emotion => {
    const meta = getEmotionMeta(emotion);
    const chip = document.createElement('div');
    chip.className = 'emotion-chip';
    chip.style.borderColor = meta.color + '40';
    chip.innerHTML = `
      <span class="emotion-chip-icon">${meta.emoji}</span>
      <span style="color: ${meta.color}">${emotion}</span>
    `;
    emotionsGrid.appendChild(chip);
  });
}


// ── Recording ─────────────────────────────────────────────────
async function startRecording() {
  // Check browser support
  if (!navigator.mediaDevices || !navigator.mediaDevices.getUserMedia) {
    showError('Browser Not Supported', 'Your browser does not support microphone access. Try Chrome, Edge, or Firefox.');
    return;
  }

  try {
    // Request microphone access
    const stream = await navigator.mediaDevices.getUserMedia({
      audio: {
        channelCount: 1,      // Mono
        sampleRate: 22050,    // Match training sample rate if possible
        echoCancellation: true,
        noiseSuppression: true,
      }
    });
    currentStream = stream;

    // Choose best supported MIME type
    const mimeTypes = [
      'audio/webm;codecs=opus',
      'audio/webm',
      'audio/ogg;codecs=opus',
      'audio/ogg',
```

```
      'audio/wav',
    ];
    let mimeType = '';
    for (const type of mimeTypes) {
      if (MediaRecorder.isTypeSupported(type)) {
        mimeType = type;
        break;
      }
    }

    audioChunks = [];
    const options = mimeType ? { mimeType } : {};
    mediaRecorder = new MediaRecorder(stream, options);

    mediaRecorder.ondataavailable = (event) => {
      if (event.data && event.data.size > 0) {
        audioChunks.push(event.data);
      }
    };

    mediaRecorder.onstop = async () => {
      stopStream();
      await processRecording(mimeType || 'audio/webm');
    };

    mediaRecorder.onerror = (event) => {
      console.error('MediaRecorder error:', event.error);
      showError('Recording Error', 'An error occurred during recording. Please try again.');
    };

    // Start recording, collect data every 500ms
    mediaRecorder.start(500);
    recordingStartTime = Date.now();
    showState('recording');
    startTimerUI();

  } catch (err) {
    handleMicError(err);
  }
}


function stopRecording() {
  if (!mediaRecorder || mediaRecorder.state === 'inactive') return;

  const elapsed = Date.now() - recordingStartTime;
  if (elapsed < MIN_RECORDING_MS) {
    // Too short — wait a tiny bit
    setTimeout(() => {
      if (mediaRecorder && mediaRecorder.state !== 'inactive') {
        mediaRecorder.stop();
      }
    }, MIN_RECORDING_MS - elapsed);
    return;
  }

  mediaRecorder.stop();
  stopTimerUI();
}


function stopStream() {
  if (currentStream) {
    currentStream.getTracks().forEach(track => track.stop());
    currentStream = null;
  }
}


function handleMicError(err) {
  stopStream();
  let title = 'Microphone Error';
  let msg = 'Could not access microphone.';

  if (err.name === 'NotAllowedError' || err.name === 'PermissionDeniedError') {
    title = 'Microphone Permission Denied';
    msg = 'Please allow microphone access in your browser settings, then try again.';
  } else if (err.name === 'NotFoundError' || err.name === 'DevicesNotFoundError') {
    title = 'No Microphone Found';
    msg = 'No microphone was detected. Please connect a microphone and try again.';
  } else if (err.name === 'NotReadableError') {
    title = 'Microphone Busy';
    msg = 'The microphone is being used by another application. Please close it and try again.';
  } else if (err.name === 'SecurityError') {
    title = 'Security Error';
    msg = 'Microphone access is blocked. Make sure you are on a secure connection (HTTPS or localhost).';
```

```
  }

  showError(title, msg);
}


// ── Timer UI ──────────────────────────────────────────────────
function startTimerUI() {
  recordingStartTime = recordingStartTime || Date.now();
  recordTimer.textContent = '00:00';
  recordingTimer = setInterval(() => {
    const elapsed = Math.floor((Date.now() - recordingStartTime) / 1000);
    const mm = String(Math.floor(elapsed / 60)).padStart(2, '0');
    const ss = String(elapsed % 60).padStart(2, '0');
    recordTimer.textContent = `${mm}:${ss}`;
  }, 500);
}

function stopTimerUI() {
  if (recordingTimer) { clearInterval(recordingTimer); recordingTimer = null; }
}


// ── Process Recording ─────────────────────────────────────────
async function processRecording(mimeType) {
  if (audioChunks.length === 0) {
    showError('Empty Recording', 'No audio was captured. Please try again.');
    return;
  }

  // Create audio blob
  const audioBlob = new Blob(audioChunks, { type: mimeType || 'audio/webm' });

  if (audioBlob.size < 1000) {
    showError('Recording Too Short', 'The recording was too short. Please speak for at least 1 second.');
    return;
  }

  showState('processing');

  // Determine file extension from MIME type
  let ext = 'webm';
  if (mimeType.includes('ogg'))  ext = 'ogg';
  else if (mimeType.includes('wav')) ext = 'wav';
  else if (mimeType.includes('mp4')) ext = 'mp4';

  const formData = new FormData();
  formData.append('audio', audioBlob, `recording.${ext}`);

  try {
    const response = await fetch(`${API_BASE}/predict`, {
      method: 'POST',
      body: formData,
      signal: AbortSignal.timeout(30000),  // 30s timeout
    });

    if (!response.ok) {
      let detail = 'Prediction failed.';
      try {
        const errData = await response.json();
        detail = errData.detail || detail;
      } catch (_) {}

      if (response.status === 503) {
        showError('Model Not Ready', 'The ML model is not loaded. Please run training first: python src/train.py');
      } else if (response.status === 400) {
        showError('Audio Issue', detail);
      } else {
        showError('Prediction Failed', detail);
      }
      return;
    }

    const result = await response.json();
    displayResult(result);
    saveToHistory(result);

  } catch (err) {
    if (err.name === 'TimeoutError' || err.name === 'AbortError') {
      showError('Request Timeout', 'The server took too long to respond. Please try a shorter recording.');
    } else if (err.name === 'TypeError' && err.message.includes('fetch')) {
      showError('Backend Offline', 'Cannot reach the backend server. Make sure it is running on port 8000.');
    } else {
      showError('Network Error', `${err.message}. Please check your connection and try again.`);
    }
  }
```

```
}


// ── Display Results ──────────────────────────────────────────
function displayResult(data) {
  const { emotion, confidence, probabilities } = data;
  const meta = getEmotionMeta(emotion);

  // Emotion icon + label
  resultEmoji.textContent = meta.emoji;
  resultEmotion.textContent = emotion.toUpperCase();
  resultEmotion.style.background = `linear-gradient(135deg, ${meta.color}, #a855f7)`;
  resultEmotion.style.webkitBackgroundClip = 'text';
  resultEmotion.style.backgroundClip = 'text';
  resultEmotion.style.webkitTextFillColor = 'transparent';

  // Confidence bar (animate after slight delay)
  const confPct = Math.round(confidence * 100);
  confValue.textContent = `${confPct}%`;
  setTimeout(() => {
    confBar.style.width = `${confPct}%`;
  }, 100);

  // Probability bars
  renderProbabilityBars(probabilities, emotion);

  showState('result');
}


function renderProbabilityBars(probabilities, topEmotion) {
  probBars.innerHTML = '';

  // Sort by probability descending
  const sorted = Object.entries(probabilities)
    .sort(([, a], [, b]) => b - a);

  sorted.forEach(([emotion, prob], index) => {
    const meta = getEmotionMeta(emotion);
    const pct = Math.round(prob * 100);
    const isTop = emotion === topEmotion;

    const row = document.createElement('div');
    row.className = 'prob-row';
    row.innerHTML = `
      <span class="prob-emotion-name">${meta.emoji} ${emotion}</span>
      <div class="prob-bar-wrap">
        <div class="prob-bar-fill"
             style="background: ${meta.color}; --delay: ${index * 0.05}s; opacity: ${isTop ? 1 : 0.65};"
             data-target="${pct}"></div>
      </div>
      <span class="prob-pct ${isTop ? 'top' : ''}">${pct}%</span>
    `;
    probBars.appendChild(row);

    // Animate bar width
    setTimeout(() => {
      const bar = row.querySelector('.prob-bar-fill');
      if (bar) bar.style.width = `${pct}%`;
    }, 150 + index * 50);
  });
}


// ── Error Display ────────────────────────────────────────────
function showError(title, message) {
  errorTitle.textContent = title;
  errorMessage.textContent = message;
  stopStream();
  stopTimerUI();
  showState('error');
}


// ── Prediction History (localStorage) ────────────────────────
function saveToHistory(data) {
  const history = getHistory();
  const entry = {
    id:         Date.now(),
    emotion:    data.emotion,
    confidence: data.confidence,
    timestamp:  new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
    fullTime:   new Date().toLocaleString(),
  };
  history.unshift(entry);
  if (history.length > MAX_HISTORY) history.splice(MAX_HISTORY);
```

```
  localStorage.setItem(HISTORY_KEY, JSON.stringify(history));
  renderHistory();
}

function getHistory() {
  try {
    return JSON.parse(localStorage.getItem(HISTORY_KEY) || '[]');
  } catch { return []; }
}

function clearHistory() {
  localStorage.removeItem(HISTORY_KEY);
  renderHistory();
}

function renderHistory() {
  const history = getHistory();
  const items = historyList.querySelectorAll('.history-item');
  items.forEach(el => el.remove());

  if (history.length === 0) {
    historyEmpty.style.display = '';
    return;
  }

  historyEmpty.style.display = 'none';

  history.forEach(entry => {
    const meta = getEmotionMeta(entry.emotion);
    const pct = Math.round(entry.confidence * 100);
    const item = document.createElement('div');
    item.className = 'history-item';
    item.style.borderLeft = `3px solid ${meta.color}`;
    item.innerHTML = `
      <span class="history-emoji">${meta.emoji}</span>
      <div class="history-info">
        <div class="history-emotion" style="color: ${meta.color}">${entry.emotion}</div>
        <div class="history-time">${entry.fullTime || entry.timestamp}</div>
      </div>
      <span class="history-conf">${pct}%</span>
    `;
    historyList.appendChild(item);
  });
}


// ── Event Listeners ──────────────────────────────────────────
startBtn.addEventListener('click', () => startRecording());
stopBtn.addEventListener('click', () => stopRecording());
retryBtn.addEventListener('click', () => showState('idle'));
recordAgainBtn.addEventListener('click', () => {
  audioChunks = [];
  showState('idle');
});
clearHistoryBtn.addEventListener('click', () => {
  if (confirm('Clear all prediction history?')) clearHistory();
});

// Smooth scroll nav
document.querySelectorAll('.nav-link').forEach(link => {
  link.addEventListener('click', (e) => {
    e.preventDefault();
    const target = link.getAttribute('href');
    const section = document.querySelector(target);
    if (section) section.scrollIntoView({ behavior: 'smooth', block: 'start' });

    document.querySelectorAll('.nav-link').forEach(l => l.classList.remove('active'));
    link.classList.add('active');
  });
});

// Update active nav on scroll
const sections = ['recorder', 'history', 'about'];
const navLinks = { recorder: document.getElementById('nav-recorder'), history: document.getElementById('nav-history'), about: d
window.addEventListener('scroll', () => {
  let current = 'recorder';
  sections.forEach(id => {
    const el = document.getElementById(id);
    if (el && window.scrollY >= el.offsetTop - 100) current = id;
  });
  Object.values(navLinks).forEach(l => l && l.classList.remove('active'));
  if (navLinks[current]) navLinks[current].classList.add('active');
}, { passive: true });


// ── Init ─────────────────────────────────────────────────────
```

```
525 (function init() {
526   showState('idle');
      renderHistory();
      checkHealth();

      // Re-check health every 30 seconds
      setInterval(checkHealth, 30000);

      // Load emotions grid from API
      fetch(`${API_BASE}/emotions`)
        .then(r => r.json())
        .then(data => { if (data.emotions) renderEmotionsGrid(data.emotions); })
        .catch(() => {
          // If API offline, show default emotions
          renderEmotionsGrid(Object.keys(EMOTION_META));
        });

      console.log('🎙️ Speech Emotion Detection — Frontend loaded');
      console.log(`API: ${API_BASE}`);
    })();
```

5. Testing & Verification Suite

Automated unit and integration test coverage for signal processing and ML pipeline (tests/)

PY **tests/test_pipeline.py** 544 Lines • 22,374 bytes

Description & Role: Comprehensive pytest automated test suite. Validates audio file loading, duration trimming and padding, data augmentation integrity, feature extraction output dimensions (verifying exact 182-feature vector), model inference accuracy, label encoding consistency, and FastAPI backend HTTP endpoint responses.

```python
"""
test_pipeline.py - Automated tests for the Speech Emotion Detection system.

Tests cover:
1. Audio preprocessing
2. Feature extraction
3. Model loading
4. Prediction pipeline
5. Backend API endpoints

Run with:
    python -m pytest tests/ -v
    or
    python tests/test_pipeline.py
"""

import sys
import os
import json
import struct
import wave
import pytest
import numpy as np
from pathlib import Path
from io import BytesIO

# Setup path
PROJECT_ROOT = Path(__file__).resolve().parent.parent
SRC_DIR = PROJECT_ROOT / "src"
BACKEND_DIR = PROJECT_ROOT / "backend"
sys.path.insert(0, str(SRC_DIR))
sys.path.insert(0, str(BACKEND_DIR))

# ── Helpers ──────────────────────────────────────────────────────

def generate_sine_wave_wav(frequency=440, duration=3.0, sample_rate=22050) -> bytes:
    """Generate a WAV file containing a sine wave — used for testing without real audio."""
    t = np.linspace(0, duration, int(sample_rate * duration), endpoint=False)
    audio = (np.sin(2 * np.pi * frequency * t) * 0.5 * 32767).astype(np.int16)

    buffer = BytesIO()
    with wave.open(buffer, 'wb') as wf:
        wf.setnchannels(1)          # Mono
        wf.setsampwidth(2)          # 16-bit
        wf.setframerate(sample_rate)
        wf.writeframes(audio.tobytes())
    return buffer.getvalue()


def generate_silent_wav(duration=0.1, sample_rate=22050) -> bytes:
    """Generate a nearly silent WAV file."""
    n_samples = int(sample_rate * duration)
    audio = np.zeros(n_samples, dtype=np.int16)

    buffer = BytesIO()
    with wave.open(buffer, 'wb') as wf:
        wf.setnchannels(1)
        wf.setsampwidth(2)
        wf.setframerate(sample_rate)
        wf.writeframes(audio.tobytes())
    return buffer.getvalue()


def save_temp_wav(content: bytes) -> str:
    """Save bytes to a temp WAV file and return its path."""
    import tempfile
    with tempfile.NamedTemporaryFile(suffix='.wav', delete=False) as f:
        f.write(content)
        return f.name


# ═══════════════════════════════════════════════════════════════
# TEST GROUP 1: Audio Preprocessing
# ═══════════════════════════════════════════════════════════════

class TestPreprocessing:
```

```
    """Tests for src/preprocessing.py"""

    def test_load_audio_valid(self):
        """Should load a valid WAV file successfully."""
        from preprocessing import load_audio

        wav_bytes = generate_sine_wave_wav(frequency=440, duration=2.0)
        tmp_path = save_temp_wav(wav_bytes)
        try:
            y, sr = load_audio(tmp_path, target_sr=22050)
            assert y is not None, "Audio should not be None"
            assert len(y) > 0, "Audio array should not be empty"
            assert sr == 22050, f"Sample rate should be 22050, got {sr}"
        finally:
            os.unlink(tmp_path)

    def test_load_audio_nonexistent(self):
        """Should return (None, 0) for missing file."""
        from preprocessing import load_audio

        y, sr = load_audio("/nonexistent/path/audio.wav")
        assert y is None, "Should return None for missing file"
        assert sr == 0

    def test_validate_audio_valid(self):
        """Valid audio should pass validation."""
        from preprocessing import validate_audio

        y = np.sin(2 * np.pi * 440 * np.linspace(0, 3, 66150)).astype(np.float32)
        is_valid, reason = validate_audio(y, sr=22050)
        assert is_valid, f"Valid audio failed validation: {reason}"
        assert reason == "OK"

    def test_validate_audio_too_short(self):
        """Audio below min duration should fail validation."""
        from preprocessing import validate_audio

        y = np.zeros(100, dtype=np.float32)  # ~0.005s at 22050 Hz
        y[50] = 0.5  # Not silent
        is_valid, reason = validate_audio(y, sr=22050, min_duration=0.3)
        assert not is_valid, "Very short audio should fail validation"

    def test_validate_audio_silent(self):
        """Silent audio should fail validation."""
        from preprocessing import validate_audio

        y = np.zeros(22050, dtype=np.float32)  # 1 second of silence
        is_valid, reason = validate_audio(y, sr=22050)
        assert not is_valid, "Silent audio should fail validation"

    def test_normalize_audio(self):
        """Normalized audio should have max amplitude of 1.0."""
        from preprocessing import normalize_audio

        y = np.array([0.2, 0.5, -0.3, 0.8, -0.6], dtype=np.float32)
        y_norm = normalize_audio(y)
        assert abs(np.max(np.abs(y_norm)) - 1.0) < 1e-5, "Max amplitude should be 1.0 after normalization"

    def test_pad_or_trim_padding(self):
        """Short audio should be zero-padded to target duration."""
        from preprocessing import pad_or_trim

        y = np.ones(11025, dtype=np.float32)  # 0.5s at 22050 Hz
        y_padded = pad_or_trim(y, sr=22050, duration=3.0)
        expected_len = int(22050 * 3.0)
        assert len(y_padded) == expected_len, f"Padded length should be {expected_len}, got {len(y_padded)}"
        assert np.all(y_padded[11025:] == 0), "Padded portion should be zeros"

    def test_pad_or_trim_trimming(self):
        """Long audio should be trimmed to target duration."""
        from preprocessing import pad_or_trim

        y = np.ones(100000, dtype=np.float32)  # ~4.5s at 22050 Hz
        y_trimmed = pad_or_trim(y, sr=22050, duration=3.0)
        expected_len = int(22050 * 3.0)
        assert len(y_trimmed) == expected_len, f"Trimmed length should be {expected_len}"

    def test_full_preprocess_pipeline(self):
        """Full preprocessing pipeline should return correct array dimensions."""
        from preprocessing import preprocess_audio

        wav_bytes = generate_sine_wave_wav(frequency=440, duration=4.0)
        tmp_path = save_temp_wav(wav_bytes)
        try:
            y, sr = preprocess_audio(tmp_path, target_sr=22050, duration=3.0)
            assert y is not None, "Preprocessed audio should not be None"
```

```
            assert sr == 22050
            assert len(y) == int(22050 * 3.0), f"Expected {22050*3} samples, got {len(y)}"
        finally:
            os.unlink(tmp_path)

    def test_preprocess_bytes(self):
        """Preprocessing from bytes should work correctly."""
        from preprocessing import preprocess_audio_bytes

        wav_bytes = generate_sine_wave_wav(frequency=440, duration=3.0)
        y, sr = preprocess_audio_bytes(wav_bytes, original_format='wav')
        assert y is not None
        assert sr == 22050
        assert len(y) == int(22050 * 3.0)


# ============================================================
# TEST GROUP 2: Feature Extraction
# ============================================================

class TestFeatureExtraction:
    """Tests for src/feature_extraction.py"""

    def test_feature_dimension(self):
        """Feature vector should have the expected dimension."""
        from feature_extraction import get_feature_dimension

        dim = get_feature_dimension()
        assert dim > 0, "Feature dimension must be positive"
        # With current config: 80 + 80 + 2 + 2 + 2 + 2 + 24 = 192
        assert dim == 192, f"Expected 192 features, got {dim}"

    def test_extract_features_from_waveform(self):
        """Feature extraction from a waveform should return correct shape."""
        from feature_extraction import extract_features_from_waveform, get_feature_dimension

        y = np.sin(2 * np.pi * 440 * np.linspace(0, 3, 66150)).astype(np.float32)
        features = extract_features_from_waveform(y, sr=22050)

        assert features is not None, "Features should not be None"
        expected_dim = get_feature_dimension()
        assert len(features) == expected_dim, f"Expected {expected_dim} features, got {len(features)}"

    def test_features_are_finite(self):
        """Feature vector should not contain NaN or Inf values."""
        from feature_extraction import extract_features_from_waveform

        y = np.sin(2 * np.pi * 440 * np.linspace(0, 3, 66150)).astype(np.float32)
        features = extract_features_from_waveform(y, sr=22050)

        assert features is not None
        assert np.all(np.isfinite(features)), "Features contain NaN or Inf values"

    def test_extract_features_from_file(self):
        """extract_features() should work end-to-end on a WAV file."""
        from feature_extraction import extract_features, get_feature_dimension

        wav_bytes = generate_sine_wave_wav(duration=3.0)
        tmp_path = save_temp_wav(wav_bytes)
        try:
            features = extract_features(tmp_path)
            assert features is not None, "Features should not be None"
            assert len(features) == get_feature_dimension()
        finally:
            os.unlink(tmp_path)

    def test_extract_features_from_bytes(self):
        """extract_features_from_bytes() should work for API use case."""
        from feature_extraction import extract_features_from_bytes, get_feature_dimension

        wav_bytes = generate_sine_wave_wav(duration=3.0)
        features = extract_features_from_bytes(wav_bytes, 'wav')

        assert features is not None
        assert len(features) == get_feature_dimension()

    def test_feature_consistency(self):
        """Same audio → same features (deterministic)."""
        from feature_extraction import extract_features_from_waveform

        y = np.sin(2 * np.pi * 440 * np.linspace(0, 3, 66150)).astype(np.float32)
        f1 = extract_features_from_waveform(y, sr=22050)
        f2 = extract_features_from_waveform(y, sr=22050)

        assert f1 is not None and f2 is not None
        np.testing.assert_array_almost_equal(f1, f2, decimal=5,
```

```
            err_msg="Feature extraction is not deterministic")

    def test_different_emotions_different_features(self):
        """Different audio signals should produce different feature vectors."""
        from feature_extraction import extract_features_from_waveform

        t = np.linspace(0, 3, 66150)
        y_low  = (np.sin(2 * np.pi * 200 * t) * 0.3).astype(np.float32)   # Low pitch, quiet
        y_high = (np.sin(2 * np.pi * 2000 * t) * 0.9).astype(np.float32)  # High pitch, loud

        f1 = extract_features_from_waveform(y_low, sr=22050)
        f2 = extract_features_from_waveform(y_high, sr=22050)

        assert f1 is not None and f2 is not None
        assert not np.allclose(f1, f2), "Different audio should produce different features"


# ═══════════════════════════════════════════════════════════════
# TEST GROUP 3: Model
# ═══════════════════════════════════════════════════════════════

class TestModel:
    """Tests for model loading and prediction (requires trained model)."""

    MODEL_AVAILABLE = (PROJECT_ROOT / "models" / "speech_emotion_model.pkl").exists()

    def test_model_files_exist(self):
        """All required model files should exist after training."""
        if not self.MODEL_AVAILABLE:
            pytest.skip("Model not yet trained. Run: python src/train.py")

        from config import MODEL_PATH, SCALER_PATH, LABEL_ENCODER_PATH, METADATA_PATH
        assert MODEL_PATH.exists(), f"Model file missing: {MODEL_PATH}"
        assert SCALER_PATH.exists(), f"Scaler missing: {SCALER_PATH}"
        assert LABEL_ENCODER_PATH.exists(), f"Label encoder missing: {LABEL_ENCODER_PATH}"
        assert METADATA_PATH.exists(), f"Metadata missing: {METADATA_PATH}"

    def test_model_loads_correctly(self):
        """Model should load without errors."""
        if not self.MODEL_AVAILABLE:
            pytest.skip("Model not yet trained. Run: python src/train.py")

        import joblib
        from config import MODEL_PATH, SCALER_PATH, LABEL_ENCODER_PATH

        model = joblib.load(MODEL_PATH)
        scaler = joblib.load(SCALER_PATH)
        label_encoder = joblib.load(LABEL_ENCODER_PATH)

        assert model is not None
        assert scaler is not None
        assert label_encoder is not None
        assert len(label_encoder.classes_) >= 2, "Should have at least 2 emotion classes"

    def test_model_metadata(self):
        """Model metadata should contain required fields."""
        if not self.MODEL_AVAILABLE:
            pytest.skip("Model not yet trained. Run: python src/train.py")

        from config import METADATA_PATH

        with open(METADATA_PATH) as f:
            metadata = json.load(f)

        required_keys = ['model_type', 'dataset', 'emotion_classes', 'test_accuracy', 'feature_dimension']
        for key in required_keys:
            assert key in metadata, f"Metadata missing key: {key}"

        assert metadata['test_accuracy'] > 0, "Test accuracy should be positive"
        assert metadata['feature_dimension'] == 192, "Feature dimension should be 192"
        assert len(metadata['emotion_classes']) >= 2

    def test_prediction_on_synthetic_audio(self):
        """Model should return a valid prediction on synthetic audio."""
        if not self.MODEL_AVAILABLE:
            pytest.skip("Model not yet trained. Run: python src/train.py")

        import joblib
        from config import MODEL_PATH, SCALER_PATH, LABEL_ENCODER_PATH
        from feature_extraction import extract_features_from_waveform

        model = joblib.load(MODEL_PATH)
        scaler = joblib.load(SCALER_PATH)
        label_encoder = joblib.load(LABEL_ENCODER_PATH)

        # Generate test audio
```

```
        y = np.sin(2 * np.pi * 440 * np.linspace(0, 3, 66150)).astype(np.float32)
        features = extract_features_from_waveform(y, sr=22050)

        assert features is not None
        features_scaled = scaler.transform(features.reshape(1, -1))
        prediction = model.predict(features_scaled)[0]

        assert prediction in label_encoder.classes_, \
            f"Prediction '{prediction}' not in known classes {label_encoder.classes_}"

    def test_prediction_returns_probabilities(self):
        """Model should return probabilities for all emotion classes."""
        if not self.MODEL_AVAILABLE:
            pytest.skip("Model not yet trained. Run: python src/train.py")

        import joblib
        from config import MODEL_PATH, SCALER_PATH, LABEL_ENCODER_PATH
        from feature_extraction import extract_features_from_waveform

        model = joblib.load(MODEL_PATH)
        scaler = joblib.load(SCALER_PATH)
        label_encoder = joblib.load(LABEL_ENCODER_PATH)

        y = np.sin(2 * np.pi * 440 * np.linspace(0, 3, 66150)).astype(np.float32)
        features = extract_features_from_waveform(y, sr=22050)
        features_scaled = scaler.transform(features.reshape(1, -1))

        proba = model.predict_proba(features_scaled)[0]

        assert len(proba) == len(label_encoder.classes_), \
            "Probability array length should match number of classes"
        assert abs(sum(proba) - 1.0) < 0.01, "Probabilities should sum to ~1.0"
        assert all(0 <= p <= 1 for p in proba), "All probabilities should be in [0, 1]"


# ═══════════════════════════════════════════════════════════════
# TEST GROUP 4: Backend API
# ═══════════════════════════════════════════════════════════════

class TestBackendAPI:
    """Tests for the FastAPI backend (requires server to be running or uses TestClient)."""

    @pytest.fixture
    def client(self):
        """FastAPI test client — no server needed."""
        try:
            from fastapi.testclient import TestClient
            sys.path.insert(0, str(BACKEND_DIR))
            from main import app, load_model
            load_model()
            with TestClient(app) as test_client:
                yield test_client
        except ImportError:
            pytest.skip("httpx not installed for TestClient")

    def test_health_endpoint(self, client):
        """/health should return 200."""
        resp = client.get("/health")
        assert resp.status_code == 200
        data = resp.json()
        assert "status" in data
        assert "model_loaded" in data

    def test_emotions_endpoint(self, client):
        """/emotions should return emotion list (if model loaded)."""
        resp = client.get("/emotions")
        if resp.status_code == 503:
            pytest.skip("Model not loaded — run training first")
        assert resp.status_code == 200
        data = resp.json()
        assert "emotions" in data
        assert len(data["emotions"]) >= 2

    def test_model_info_endpoint(self, client):
        """/model-info should return metadata."""
        resp = client.get("/model-info")
        if resp.status_code == 503:
            pytest.skip("Model not loaded — run training first")
        assert resp.status_code == 200

    def test_predict_valid_audio(self, client):
        """/predict with valid WAV should return emotion prediction."""
        MODEL_AVAILABLE = (PROJECT_ROOT / "models" / "speech_emotion_model.pkl").exists()
        if not MODEL_AVAILABLE:
            pytest.skip("Model not trained yet. Run: python src/train.py")
```

```
        wav_bytes = generate_sine_wave_wav(duration=3.0)
        files = {'audio': ('test.wav', wav_bytes, 'audio/wav')}
        resp = client.post("/predict", files=files)

        assert resp.status_code == 200, f"Expected 200, got {resp.status_code}: {resp.text}"
        data = resp.json()
        assert "emotion" in data
        assert "confidence" in data
        assert "probabilities" in data
        assert 0.0 <= data["confidence"] <= 1.0
        assert len(data["probabilities"]) >= 2
        assert abs(sum(data["probabilities"].values()) - 1.0) < 0.05

    def test_predict_empty_audio(self, client):
        """/predict with empty file should return 400."""
        files = {'audio': ('empty.wav', b'', 'audio/wav')}
        resp = client.post("/predict", files=files)
        assert resp.status_code in [400, 422, 503], \
            f"Empty audio should return error, got {resp.status_code}"

    def test_predict_invalid_data(self, client):
        """/predict with random bytes should handle gracefully (not crash)."""
        files = {'audio': ('garbage.wav', b'not audio data at all!#@$', 'audio/wav')}
        resp = client.post("/predict", files=files)
        assert resp.status_code in [400, 422, 500, 503], \
            f"Invalid audio should return error, not {resp.status_code}"

    def test_predict_short_audio(self, client):
        """/predict with very short audio may return 400."""
        MODEL_AVAILABLE = (PROJECT_ROOT / "models" / "speech_emotion_model.pkl").exists()
        if not MODEL_AVAILABLE:
            pytest.skip("Model not trained")

        wav_bytes = generate_silent_wav(duration=0.05)  # 50ms — very short
        files = {'audio': ('short.wav', wav_bytes, 'audio/wav')}
        resp = client.post("/predict", files=files)
        # Either 400 (too short) or 422 (feature extraction failed) is acceptable
        assert resp.status_code in [200, 400, 422, 503], \
            f"Unexpected status: {resp.status_code}"


# ═══════════════════════════════════════════════════════════════════
# TEST GROUP 5: Dataset
# ═══════════════════════════════════════════════════════════════════

class TestDataset:
    """Tests for dataset loading (RAVDESS must be present)."""

    DATASET_AVAILABLE = (PROJECT_ROOT / "data" / "raw" / "RAVDESS").exists() and \
                        any((PROJECT_ROOT / "data" / "raw" / "RAVDESS").rglob("*.wav"))

    def test_parse_ravdess_filename_valid(self):
        """Valid RAVDESS filename should parse correctly."""
        from dataset import parse_ravdess_filename

        # 03-01-05-01-01-01-12.wav → Actor 12, Emotion 05 (Angry), speech
        path = Path("03-01-05-01-01-01-12.wav")
        result = parse_ravdess_filename(path)

        assert result is not None, "Should parse valid RAVDESS filename"
        assert result['emotion'] == 'Angry'
        assert result['actor_id'] == 12
        assert result['emotion_code'] == '05'

    def test_parse_ravdess_filename_song(self):
        """Song files (vocal_channel=02) should be skipped."""
        from dataset import parse_ravdess_filename

        # 03-02-05-01-01-01-12.wav → song (vocal_channel=02)
        path = Path("03-02-05-01-01-01-12.wav")
        result = parse_ravdess_filename(path)
        assert result is None, "Song files should return None"

    def test_parse_ravdess_all_emotions(self):
        """All 8 emotion codes should parse correctly."""
        from dataset import parse_ravdess_filename
        from config import RAVDESS_EMOTION_MAP

        for code, expected_label in RAVDESS_EMOTION_MAP.items():
            path = Path(f"03-01-{code}-01-01-01-01.wav")
            result = parse_ravdess_filename(path)
            assert result is not None, f"Failed to parse emotion code {code}"
            assert result['emotion'] == expected_label, \
                f"Expected {expected_label}, got {result['emotion']}"

    def test_dataset_not_found_returns_empty(self):
```

```
        """Loading from non-existent directory should return empty DataFrame."""
        from dataset import load_dataset_info
        from pathlib import Path

        df = load_dataset_info(Path("/nonexistent/path"))
        assert df.empty, "Should return empty DataFrame when dataset not found"

    def test_speaker_split_no_overlap(self):
        """Train, val, and test speaker sets should not overlap."""
        if not self.DATASET_AVAILABLE:
            pytest.skip("RAVDESS dataset not present")

        from dataset import load_dataset_info, get_speaker_split
        from config import RAW_DIR

        df = load_dataset_info(RAW_DIR)
        if df.empty:
            pytest.skip("Dataset empty")

        train_a, val_a, test_a = get_speaker_split(df)

        train_set = set(train_a)
        val_set   = set(val_a)
        test_set  = set(test_a)

        assert not (train_set & val_set), "Train and Val speaker sets overlap!"
        assert not (train_set & test_set), "Train and Test speaker sets overlap!"
        assert not (val_set & test_set), "Val and Test speaker sets overlap!"


# ── Main runner ──────────────────────────────────────────────────────────
if __name__ == "__main__":
    import subprocess
    result = subprocess.run(
        [sys.executable, "-m", "pytest", __file__, "-v", "--tb=short"],
        cwd=str(PROJECT_ROOT)
    )
    sys.exit(result.returncode)
```

6. Model Configuration, Dependencies & Launch Scripts

Serialized model metadata, feature dimension configurations, dependencies, and runner scripts

JSON **models/model_metadata.json** 24 Lines • 522 bytes

Description & Role: JSON metadata file recording the trained model type, test accuracy, macro F1-score, feature dimension count, supported emotion classes, training timestamp, and scikit-learn version.

```json
{
  "model_type": "MLP_Wide",
  "dataset": "RAVDESS",
  "dataset_url": "https://zenodo.org/record/1188976",
  "feature_type": "MFCC+Delta+ZCR+RMS+Centroid+Rolloff+Chroma",
  "feature_dimension": 560,
  "emotion_classes": [
    "Angry",
    "Calm",
    "Disgust",
    "Fearful",
    "Happy",
    "Neutral",
    "Sad",
    "Surprised"
  ],
  "n_emotions": 8,
  "random_seed": 42,
  "training_date": "2026-09-27T20:25:06.855172",
  "test_accuracy": 0.55,
  "macro_f1": 0.5568,
  "train_samples": 4080,
  "test_samples": 240
}
```

JSON **models/features_config.json** 10 Lines • 195 bytes

Description & Role: JSON configuration defining exact feature extraction parameters, individual feature counts (MFCCs, Chroma, Mel Spectrogram, Contrast, Tonnetz, ZCR, RMS), and total feature vector dimension (182).

```json
{
  "mfcc": true,
  "mfcc_delta": true,
  "mel_spectrogram": false,
  "zero_crossing_rate": true,
  "rms_energy": true,
  "spectral_centroid": true,
  "spectral_rolloff": true,
  "chroma": true
}
```

TXT **requirements.txt** 40 Lines • 1,829 bytes

Description & Role: Python package dependencies with version constraints: librosa, soundfile, scikit-learn, numpy, pandas, matplotlib, seaborn, fastapi, uvicorn, python-multipart, pytest, requests, etc.

```
# Speech Emotion Detection — Python Dependencies
# Install with: pip install -r requirements.txt
#
# Python version: 3.10+
# Tested on: Python 3.14.3

# ── Audio processing ──────────────────────────────────────
librosa>=0.10.0           # Audio loading, MFCC, spectral features
soundfile>=0.12.0         # Fast WAV/FLAC reading
numpy>=1.24.0             # Numerical arrays
scipy>=1.10.0             # Signal processing utilities
imageio-ffmpeg>=0.4.9     # Bundled ffmpeg binary — required to decode browser WebM/Opus on Windows

# ── Machine Learning ──────────────────────────────────────
scikit-learn>=1.2.0       # SVM, Random Forest, preprocessing, metrics
joblib>=1.2.0             # Model serialization (load/save)

# ── Data analysis & visualization ─────────────────────────
pandas>=2.0.0             # DataFrame handling
matplotlib>=3.7.0         # Plots and charts
seaborn>=0.12.0           # Enhanced visualization

# ── Backend (FastAPI) ─────────────────────────────────────
fastapi>=0.100.0          # Web framework
uvicorn>=0.22.0           # ASGI server
python-multipart>=0.0.6   # File upload support
aiofiles>=23.0.0          # Async file handling
pydub>=0.25.0             # Audio format conversion (for WebM handling)

# ── HTTP client (for tests) ───────────────────────────────
httpx>=0.24.0             # HTTP client (used by FastAPI TestClient)
requests>=2.31.0          # Simple HTTP requests

# ── Testing ───────────────────────────────────────────────
pytest>=7.4.0             # Test framework
pytest-asyncio>=0.21.0    # Async test support

# ── Optional (for report generation) ──────────────────────
# reportlab>=4.0.0        # PDF generation (uncomment if needed)
# python-docx>=0.8.11     # Word document generation (uncomment if needed)
```

PY **start.py** 193 Lines • 5,374 bytes

Description & Role: Unified Python launcher script. Validates environment dependencies, verifies dataset and trained model availability, launches FastAPI backend with Uvicorn, and automatically opens the frontend in the default web browser.

```python
"""
start.py - Cross-platform launcher for Speech Emotion Detection System.

Usage:
    python start.py setup      # Install dependencies + check dataset
    python start.py dataset    # Check/setup dataset
    python start.py train      # Train ML model
    python start.py backend    # Start FastAPI backend
    python start.py frontend   # Open frontend in browser
    python start.py test       # Run tests
    python start.py all        # Train (if needed) + backend + frontend
"""

import sys
import os
import subprocess
import webbrowser
import time
from pathlib import Path

if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8', errors='replace')
if hasattr(sys.stderr, 'reconfigure'):
    sys.stderr.reconfigure(encoding='utf-8', errors='replace')

PROJECT_ROOT = Path(__file__).resolve().parent
SRC_DIR      = PROJECT_ROOT / "src"
BACKEND_DIR  = PROJECT_ROOT / "backend"
FRONTEND_DIR = PROJECT_ROOT / "frontend"
MODEL_PATH   = PROJECT_ROOT / "models" / "speech_emotion_model.pkl"

PYTHON = sys.executable


def run(cmd: list, **kwargs):
    """Run a subprocess command."""
    print(f"\n> {' '.join(cmd)}\n")
    return subprocess.run(cmd, **kwargs)


def cmd_setup():
    """Install all dependencies and check dataset."""
    print("=" * 60)
    print("  STEP 1: Installing dependencies")
    print("=" * 60)
    run([PYTHON, "-m", "pip", "install", "-r", str(PROJECT_ROOT / "requirements.txt")])

    print("\n" + "=" * 60)
    print("  STEP 2: Checking dataset")
    print("=" * 60)
    run([PYTHON, str(SRC_DIR / "download_dataset.py")])


def cmd_dataset():
    """Check dataset status."""
    run([PYTHON, str(SRC_DIR / "download_dataset.py")])


def cmd_train():
    """Run the training pipeline."""
    print("=" * 60)
    print("  Training ML Model — This may take 10-30 minutes")
    print("=" * 60)
    result = run([PYTHON, str(SRC_DIR / "train.py")])
    if result.returncode != 0:
        print("\nERROR: Training failed. Check the output above.")
        sys.exit(1)
    print("\n✅ Training complete! You can now start the backend.")


def cmd_backend():
    """Start the FastAPI backend server."""
    print("=" * 60)
    print("  Starting FastAPI Backend")
    print("  URL: http://127.0.0.1:8000")
    print("  API docs: http://127.0.0.1:8000/docs")
    print("  Press Ctrl+C to stop")
    print("=" * 60)
```

```
    os.environ["PYTHONPATH"] = str(SRC_DIR)
    run([PYTHON, str(BACKEND_DIR / "main.py")])


def cmd_frontend():
    """Open the frontend in the default browser."""
    url = "http://127.0.0.1:8000"
    print(f"Opening: {url}")
    webbrowser.open(url)
    print("\nNOTE: Make sure the backend is running!")
    print("      Run in another terminal: python start.py backend")


def cmd_test():
    """Run the test suite."""
    print("=" * 60)
    print("  Running Test Suite")
    print("=" * 60)
    result = run([
        PYTHON, "-m", "pytest",
        str(PROJECT_ROOT / "tests"),
        "-v", "--tb=short"
    ])
    sys.exit(result.returncode)


def cmd_all():
    """Full pipeline: train if needed, start backend, open frontend."""
    print("=" * 60)
    print("  Starting Complete Speech Emotion Detection System")
    print("=" * 60)

    # Train if model doesn't exist
    if not MODEL_PATH.exists():
        print("\nNo trained model found. Starting training...")
        cmd_train()

    # Start backend in background
    print("\nStarting backend server in background...")
    backend_process = subprocess.Popen(
        [PYTHON, str(BACKEND_DIR / "main.py")],
        env={**os.environ, "PYTHONPATH": str(SRC_DIR)}
    )

    # Wait for backend to initialize
    print("Waiting for backend to start (5 seconds)...")
    time.sleep(5)

    # Open frontend
    cmd_frontend()

    print("\n" + "=" * 60)
    print("  System Running!")
    print("  Frontend: http://127.0.0.1:8000")
    print("  Backend:  http://127.0.0.1:8000")
    print("  API docs: http://127.0.0.1:8000/docs")
    print("=" * 60)

    try:
        print("\nPress Ctrl+C to stop the backend...")
        backend_process.wait()
    except KeyboardInterrupt:
        print("\nStopping backend...")
        backend_process.terminate()


COMMANDS = {
    "setup":    cmd_setup,
    "dataset":  cmd_dataset,
    "train":    cmd_train,
    "backend":  cmd_backend,
    "frontend": cmd_frontend,
    "test":     cmd_test,
    "all":      cmd_all,
}


def print_help():
    print("""
🎙  Speech Emotion Detection — Launcher

Usage:
    python start.py <command>

Commands:
    setup      Install dependencies + check dataset
```

```
    dataset    Check/setup RAVDESS dataset
    train      Train the ML model (run once)
    backend    Start FastAPI backend (keep running)
    frontend   Open web UI in browser
    test       Run test suite
    all        Full system (train if needed + backend + frontend)

Quick Start:
    1. python start.py setup
    2. python start.py train
    3. python start.py backend   ← keep this running
    4. python start.py frontend  ← in a new terminal
""")


if __name__ == "__main__":
    if len(sys.argv) < 2:
        print_help()
        sys.exit(0)

    command = sys.argv[1].lower()
    if command in COMMANDS:
        COMMANDS[command]()
    elif command in ["-h", "--help", "help"]:
        print_help()
    else:
        print(f"Unknown command: {command}")
        print_help()
        sys.exit(1)
```

BAT **run.bat** 144 Lines • 3,665 bytes

Description & Role: Windows batch automation script. Creates and activates Python virtual environment, installs requirements, checks for trained model artifacts, runs the full training pipeline if missing, and boots the backend and frontend servers.

```
@echo off
REM ============================================================
REM  run.bat — Speech Emotion Detection System Launcher
REM  Usage: run.bat [setup|train|backend|frontend|test|all]
REM ============================================================

setlocal enabledelayedexpansion
set "PROJECT_DIR=%~dp0"
set "PYTHON=python"

echo.
echo  ============================================================
echo     ^ 🎙   Speech Emotion Detection System
echo  ============================================================
echo.

REM Default action: show menu
set "ACTION=%1"
if "%ACTION%"=="" set "ACTION=menu"

goto :%ACTION% 2>nul || goto :unknown

:menu
echo  Available commands:
echo.
echo    run.bat setup     — Install all dependencies
echo    run.bat dataset   — Check/setup the dataset
echo    run.bat train     — Train the ML model
echo    run.bat backend   — Start the FastAPI backend
echo    run.bat frontend  — Open the frontend in browser
echo    run.bat test      — Run the test suite
echo    run.bat all       — Train + start backend + open browser
echo.
echo  Quick start (after dataset is ready):
echo    1. run.bat setup
echo    2. run.bat train
echo    3. run.bat backend   (in a new terminal)
echo    4. run.bat frontend
echo.
goto :eof

:setup
echo [1/3] Installing Python dependencies...
%PYTHON% -m pip install -r "%PROJECT_DIR%requirements.txt"
if errorlevel 1 (
    echo ERROR: pip install failed. Make sure Python is installed.
    pause
    exit /b 1
)
echo.
echo [2/3] Checking dataset...
%PYTHON% "%PROJECT_DIR%src\download_dataset.py"
echo.
echo [3/3] Setup complete!
echo.
echo Next step: run.bat train
pause
goto :eof

:dataset
echo Checking RAVDESS dataset...
%PYTHON% "%PROJECT_DIR%src\download_dataset.py"
pause
goto :eof

:train
echo Starting model training...
echo This may take 10-30 minutes depending on your machine.
echo.
%PYTHON% "%PROJECT_DIR%src\train.py"
if errorlevel 1 (
    echo ERROR: Training failed. Check the error above.
    pause
    exit /b 1
)
echo.
echo Training complete! You can now start the backend.
echo Run: run.bat backend
```

```
pause
goto :eof

:backend
echo Starting FastAPI backend on http://127.0.0.1:8000
echo Press Ctrl+C to stop.
echo.
echo API docs available at: http://127.0.0.1:8000/docs
echo.
%PYTHON% "%PROJECT_DIR%backend\main.py"
goto :eof

:frontend
echo Opening frontend in browser...
start "" "http://127.0.0.1:8000"
echo.
echo If the browser doesn't open automatically, navigate to:
echo http://127.0.0.1:8000
echo.
echo IMPORTANT: Make sure the backend is running first!
echo Run in another terminal: run.bat backend
pause
goto :eof

:test
echo Running test suite...
echo.
%PYTHON% -m pytest "%PROJECT_DIR%tests\" -v --tb=short
pause
goto :eof

:all
echo Starting complete system...
echo.
echo [Step 1] Checking if model exists...
if not exist "%PROJECT_DIR%models\speech_emotion_model.pkl" (
    echo Model not found. Starting training first...
    call :train
    if errorlevel 1 goto :eof
)

echo.
echo [Step 2] Starting backend in background...
start "SED Backend" cmd /c "%PYTHON% %PROJECT_DIR%backend\main.py"

echo Waiting for backend to initialize (5 seconds)...
timeout /t 5 /nobreak > nul

echo.
echo [Step 3] Opening frontend...
start "" "http://127.0.0.1:8000"

echo.
echo ============================================================
echo  System is running!
echo  Web App:  http://127.0.0.1:8000
echo  API docs: http://127.0.0.1:8000/docs
echo ============================================================
echo.
echo Close the "SED Backend" window to stop the backend.
pause
goto :eof

:unknown
echo Unknown command: %ACTION%
goto :menu
```